

When Are Agent Execution Edits Safe?

An Exact Theory of Fork, Restore, and Merge

Abstract—Agent runtimes can Fork an execution, Restore a checkpoint, or Merge branches without restarting a task. We call these operations *execution edits* because they change which parts of the workflow will run next. An edit cannot undo a tool call that has already been authorized or released. A careless edit can therefore perform the same action twice, drop work that the workflow still requires, or overlap with a call that began before the edit. We ask when an execution edit can be implemented safely. Our answer preserves every outcome that was required before the edit. Each such outcome must remain required, already have a recorded completion, or be explicitly removed by the same policy authority that required it. Existing Agent recovery systems implement particular edits, while existing synthesis methods assume that the possible executions and completion conditions are already known. Neither derives the full safety requirement of a requested edit from the execution record. We present an exact theory and an executable checker. Given a registered workflow, its authenticated record, and a requested edit, the checker constructs every complete execution allowed by the workflow and policy. It removes an execution whenever one of its partial executions would leave a still-required outcome with no safe way to finish. If nothing remains, the checker returns a finite proof that no safe implementation exists. Otherwise, the remaining executions define the most permissive runtime monitor that enforces the edit. For all six registered execution edits and registered extensions, our main theorem proves that the checker accepts exactly when such a safe implementation exists. The same theorem proves atomic installation despite racing calls, identifies the four kinds of recorded information required by any exact checker, and relates the installed runtime to an ideal atomic machine. Every supported event preserves the security invariant, so the theorem applies again after every finite reachable execution. Five kernel-checked Lean modules, 81 executable tests, and bounded performance measurements validate the finite checker, critical examples, and executable artifact.

I. INTRODUCTION

Tool-using Agent runtimes no longer follow only one linear plan. They let an Agent Fork an execution into alternatives, Restore an earlier checkpoint, and Merge branches [1], [2], [3], [4], [5], [6]. These operations support exploration, recovery, and reuse without restarting the task. We call them *execution edits* because they change which parts of the workflow will run next.

An execution edit does not undo earlier tool calls. In our running procurement example, the Agent saves checkpoint k before approval. The approval later uses its one-use permission and leaves a completion record. Restoring k creates another approval call in the workflow, but it must not authorize the same approval again. A concurrent Merge can cause a similar error by selecting supplier L after a supplier- R payment has already been authorized. An edit can therefore restore the saved program state and still be unsafe.

The problem is that a checkpoint contains only part of the information needed to decide whether an edit is safe. The controller must also know four things: which outcomes remain required, which call positions refer to the same protected action, which actions have already been authorized, and which runtime version governs a call that overlaps the edit. Each can change the correct answer. The controller cannot recover all four from equal workspaces, an execution trace alone, a receipt log alone, or a replacement workflow proposed by the Agent.

Existing approaches cover parts of this problem but do not derive its complete safety condition. Agent systems provide branching, staged effects, transactions, or recovery checks [7], [8], [9], [10], [11], [12], [13], [14]. Control and synthesis choose behavior after a model of possible executions and desired final states has been supplied [15], [16], [17], [18]. Neither line of work derives from a recorded execution everything that a particular edit must preserve before it takes effect.

Our key idea is that an execution edit must preserve both past actions and still-required outcomes. Every outcome required before the edit must remain required, already have a recorded completion, or be explicitly removed by the same policy authority that required it. The Agent selects only a registered edit. The trusted controller derives the edited workflow and the requirements it inherits from the recorded execution. The Agent therefore cannot obtain a favorable answer by submitting a replacement workflow that hides inconvenient facts.

This idea creates three technical problems. First, calls copied by Fork or Restore may refer to the same protected action, so the controller must distinguish a new action from reuse of an earlier call and return value. Second, after every allowed partial execution, every outcome that is still required and still possible must retain a safe way to finish. The checker must enforce this condition without forbidding additional safe behavior. Third, the answer must take effect atomically with respect to calls that overlap the edit or a crash.

We give an exact checker for execution edits. For a requested edit, the controller uses the registered workflow, recorded calls and edits, completion records, pending calls, policy, and current monitor version to construct every allowed completed execution. It removes an execution whenever one of its partial executions would leave a still-required outcome with no safe completion. If no execution remains, the controller returns a finite proof that no safe implementation exists. Otherwise, the remaining executions define the most permissive safe monitor. Before the edit takes effect, the runtime checks the current recorded state again and atomically replaces the old monitor. Every

overlapping call is therefore governed entirely by either the old monitor or the new one. Section II follows this process through one procurement execution containing Fork, Merge, two races, and Restore. After Restore, the repeated approval call reuses the earlier completion record instead of authorizing the approval again.

a) Contributions: We contribute one end-to-end theorem connecting the original Agent workflow, the requested execution edit, the exact checker, and the installed runtime. For all six registered execution edits and registered extensions, a safe implementation exists exactly when the checker leaves a nonempty set of executions, its monitor can be installed, and an ideal atomic machine permits the same edit. The theorem also proves that all four kinds of recorded information used by the checker are necessary and that the concrete runtime has the same visible behavior as the ideal machine (sections III to VI). Every supported event preserves the security invariant, so the theorem applies again after every finite reachable execution. Five kernel-checked Lean modules, 81 executable tests, and bounded performance measurements validate the finite checker, critical examples, and executable artifact.

II. A PROCUREMENT EDIT, END TO END

Consider a procurement Agent authorized to place one order. The Agent reserves the budget and saves checkpoint k before approval. The approval then uses its one-use permission and leaves a completion record. The Agent next requests a Fork between suppliers L and R . The workflow allows two outcomes: order from L or order from R . In either outcome, payment must precede shipment:

$$\begin{aligned} o_L : x_L^p &\prec x_L^s, \\ o_R : x_R^p &\prec x_R^s, \end{aligned} \quad (1)$$

where x_i^p and x_i^s are payment and shipment occurrences. An occurrence identifies one call position in the workflow. An effect identity identifies the protected action and one-use permission behind that position; the formulas call it a cell d .

The two shipment calls share one effect identity:

$$q_{\text{cell}}(x_L^s) = q_{\text{cell}}(x_R^s) = d_s.$$

The two payment calls instead have different effect identities d_L^p and d_R^p . A later Restore of k creates a new approval occurrence x'_a , but both approval occurrences refer to the same effect identity d_a .

The receipt log Δ records newly authorized effects in order. The notation $\text{lab}(\Delta)$ keeps only their permission labels. Budget reservation contributes r , approval contributes a , the two payments contribute p_L and p_R , and shipment contributes s . At the Fork request, $\text{lab}(\Delta) = ra$. The policy allows prefixes of rap_Ls and rap_Rs , but never both payments or a payment before approval. Although the supplier outcomes are exclusive, both must still have a safe way to finish when the Fork is checked.

a) How the checker handles the Fork: The Agent submits only $u = \text{ForkChoice}(b, \sigma)$. The trusted controller, not the Agent, constructs the edited workflow from the registered edit schema. It reads the workflow, authenticated execution record, receipt log, ordered dispatch queue, and policy, written $(\kappa, H, \Delta, \text{out}, \mathcal{A})$. From these records it derives the edited workflow, its required outcomes, and the effect identity behind every call position.

Write M_o for the policy-safe completions of outcome o . In this example, each outcome has one allowed call order:

$$\begin{aligned} z_L &= \text{fresh}(x_L^p, d_L^p) \text{fresh}(x_L^s, d_s), \\ z_R &= \text{fresh}(x_R^p, d_R^p) \text{fresh}(x_R^s, d_s). \end{aligned}$$

Thus $M_{o_L} = \{z_L\}$, $M_{o_R} = \{z_R\}$, and $\hat{B}^\dagger = \{(o_L, z_L), (o_R, z_R)\}$, with $W^\dagger$ their prefix closure. The checker builds monitor $\text{Can}(W^\dagger, \pi_2 \hat{B}^\dagger)$, binds the answer to this source snapshot, and atomically installs the monitor and replacement authorization tokens under a new monitor version. The installed monitor allows only these partial executions. For each protected call, fresh means that the effect identity receives its first receipt, while alias means that another call position reuses the existing receipt. If policy excludes rap_Rs , then $M_{o_R} = \emptyset$, and the Fork is rejected because pruning supplier R would change its meaning. Rejection returns a ranked certificate explaining which completions were removed and why, rather than letting the Agent weaken the requested edit. Section IV gives a harder case where each outcome initially has a safe completion, but every possible implementation eventually reaches a partial execution after which some still-possible outcome cannot finish safely.

b) Adding new rules without changing current requirements: A registered extension $\text{Extend}(\xi)$ can add compensation schemas and policy entries while the procurement execution remains active. The protected-call and policy authorities must both sign ξ . The extension preserves every current outcome, call position, effect identity, receipt, checkpoint, cursor, dispatch-queue entry, and causal edge. It only appends new schema and registry rows and enlarges the policy. The controller then rebuilds the monitor and atomically changes its version. A later edit may use the new schema, but the Agent cannot change an earlier call or remove a currently required outcome.

A *policy domain* is the set of protected actions governed by one policy, one receipt log, and one monitor that is replaced atomically.

c) The fresh race: After the shared approval and before either payment, the Agent requests $\text{MergeSelect}(g, L, \sigma)$. If the payment call x_R^p from the old supplier- R branch is authorized first, it selects R and changes $\text{lab}(\Delta)$ from ra to rap_R . Selecting L must therefore be re-derived and rejected. If the Merge installation occurs first, it closes the old monitor version and gives every current call a new authorization token before any later x_R^p can be authorized. Leaving the old monitor version active would allow both payments.

d) The alias race: Now consider the live Restore in figure 1. Copying checkpoint k creates a new approval occurrence x'_a with a new monitor entry and authorization

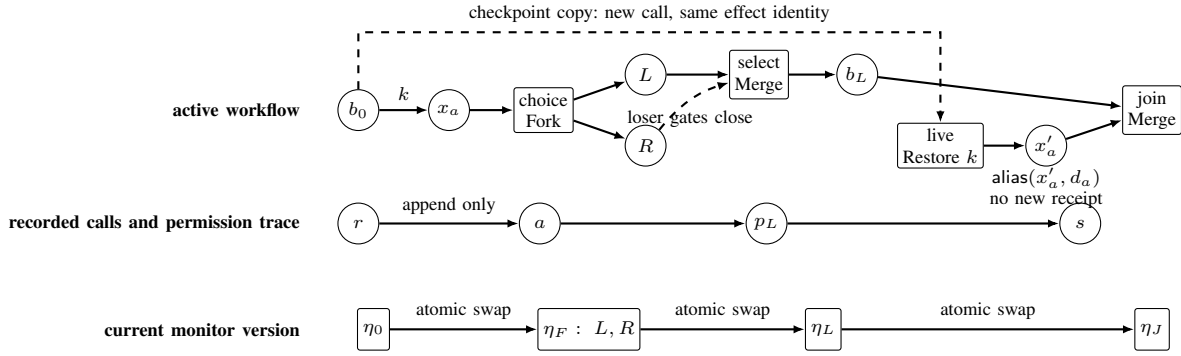


Fig. 1. Fork, Restore, and Merge change the active workflow but do not erase recorded calls. Restore creates a new call position while preserving its authenticated effect identity. The atomic installation point must order both a new authorization (fresh) and reuse of an earlier completion record (alias) against the monitor-version change.

token, but it retains effect identity d_a . Because d_a already has a receipt, executing x'_a is an alias: the restored branch advances past approval without creating another receipt. A concurrent join or replacement computed before that progress is stale even though the receipt log has not changed. The installation check must therefore compare the ordered occurrence record χ as well as the receipt log. For this RestoreLive request, the checker either installs the derived monitor under a new version or returns a ranked rejection certificate.

The example exposes the whole problem. Workspace equality does not reveal which branches remain active, and receipt equality does not reveal alias progress. Effect-identity equality alone does not reveal which outcomes the edit must preserve. A replacement monitor proposed by the Agent cannot certify any of these facts. Section III therefore derives the edited workflow from the registered schema and authenticated execution record.

III. MODEL AND EDIT DERIVATION

The model separates unrestricted internal Agent reasoning from tool calls whose effects pass through a trusted runtime monitor. Such a call must identify its registered position in the workflow, present an authorization token, and submit a concrete tool request. The monitor decides whether the call may use a new authorization, reuse an earlier completion record, or make no progress. Each modeled state covers one policy domain. Domains with disjoint protected actions, completion logs, and policies compose by product. Every state snapshot is finite, although an execution may contain an unbounded number of snapshots. The definitions below first describe the required outcomes and then the authenticated record needed to preserve them across edits.

A. Workflow outcomes and call order

a) *Typed runtime vocabulary:* We capitalize *Agent* for the untrusted principal requesting execution edits. An *occurrence* x identifies one position for a protected tool call in the workflow. An *effect identity* d , called a cell in the formulas, identifies the one protected action and one-use permission behind that position. The first authorized use of an effect identity creates a receipt. After Fork or Restore, several occurrences may

refer to the same effect identity. A monitor entry j belongs to one monitor version η . An authorization token h binds an occurrence to its monitor entry, effect identity, scope, and normalized tool request m° . *Lineage* is the certified ancestry preserved when an edit carries or copies a workflow region. T, Δ, χ denote the active workflow, receipt log, and ordered record of resolved occurrences. An edit takes effect at an atomic installation point, which also replaces the policy domain's monitor version. Registry ρ authenticates each occurrence's monitor entry and token. It maps the occurrence to $q_{\text{cell}}(x) = d$, immutable normalized request $\text{inv}(d) = m^\circ$, and permission label $\ell(d)$.

A pomset $p = (X_p, \prec_p)$ is a finite set of occurrences with a strict partial order. Its linearizations $\text{Lin}(p)$ are the words containing each member of X_p once and respecting $\prec_p$. An *indexed workflow contract* $\mathcal{C} = (O, \{p_o\}_{o \in O})$ is a finite nonempty family of pomsets. Each index o names one allowed completion outcome. Two outcomes remain distinct even if the calls still required for them later become identical. Write $X_{\mathcal{C}} = \bigcup_{o \in O} X_{p_o}$ and $\text{CL}(\mathcal{C}) = \bigcup_{o \in O} \text{Lin}(p_o)$. We require the runtime to distinguish completion from continuation.

$$v, w \in \text{CL}(\mathcal{C}) \wedge v \preceq_{\text{pref}} w \implies v = w.$$

This condition permits incomparable traces and different outcomes with the same trace, so it does not require deterministic outcomes. It requires a registered occurrence whenever continuing after a shared partial execution performs more work. An otherwise silent choice between stopping and continuing lies outside the interface. A deployment may represent such a choice with an authenticated terminal occurrence handled by the controller, without claiming that a remote action occurred.

Contracts use the three partial constructors below. Each constructor requires occurrence-disjoint operands and is defined

only when its result remains completion-observable.

$$\begin{aligned}
O_{C \oplus D} &= (\{L\} \times O_C) \uplus (\{R\} \times O_D), \\
p_{(L,o)} &= p_o, \quad p_{(R,v)} = p_v, \\
O_{C \otimes D} &= O_C \times O_D, \\
p_{(o,v)} &= p_o \uplus p_v, \\
O_{C \triangleright D} &= O_C \times O_D, \\
p_{(o,v)} &= p_o \triangleright p_v.
\end{aligned} \tag{2}$$

Before adding a registered or copied operand, the edit derivation deterministically renames its occurrences and extends ρ while retaining their authenticated effect identities. Well-formedness already makes carried operands disjoint. In equation (2), $\uplus$ adds no cross-order and $\triangleright$ orders every event of p before every event of q , while choice uses a tagged union of outcome indices and parallel and sequence use Cartesian products. Consequently, choice retains each arm's full indexed family, parallel pairs outcomes while permitting every causal linearization, and sequence adds a barrier. For raw prefix w , C/w retains each index whose pomset has a linearization prefixed by w and removes w 's occurrences and incident order, and it is undefined if none remains. This operational "remaining contract" preserves outcome and occurrence identity.

Lemma 1 (Observable completion). *Every defined residual C/w is completion-observable, and either every retained pomset is empty or no retained pomset is empty.*

Proof. If a residual suffix v strictly prefixes v' , then wv strictly prefixes wv' in $\text{CL}(C)$. If one retained pomset is empty and another is nonempty, choose a nonempty linearization v of the latter. Then w strictly prefixes wv . Both contradict completion observability. $\square$

B. Trusted history state

1) *History State and Live Frontier:* An Agent history state is

$$H = (\omega, G, T, K, \Sigma_\zeta, \rho, \beta, \chi, \nu, \eta). \tag{3}$$

ω is the policy-domain identity and η its single current policy-domain epoch. G is a finite append-only provenance store whose node identities, base contracts, and typed topology records never change. Each continuation node b carries an immutable registered base contract Γ_b , while its branch-local resolved cursor χ_b is the event projection of append-only progress records tagged by b . The immutable topology records form a DAG whose edges record typed Fork, Restore, and Merge operations. T is the live structural frontier.

$$T ::= b:C \mid T \oplus_g T \mid T \parallel_g T \mid \text{jbar}(T, T) \mid T; T.$$

$s \in \{\text{open}, L, R\}$ records an unresolved choice or the arm that made durable progress, and g groups exclusive or jointly live siblings. $\llbracket T \rrbracket$ uses both arms of an open choice and only the selected arm thereafter. It interprets parallel and jbar with $\otimes$, and sequence with $\triangleright$. The unaddressed jbar marks a joined group and prevents another Merge.

T is complete when every pomset in $\llbracket T \rrbracket$ is empty. Lemma 1 excludes a residual containing both empty and nonempty pomsets. Set $\text{heads}(b:C) = \{b\}$. Open choice exposes both arms, selected choice its winner, parallel and jbar both arms, and sequence its left operand until completion and then its right. Occurrence progress residualizes leaves without deleting structure. Because sequence exposure tests complete, it retains completed left-outcome indices, so an isolated completed leaf remains addressable for later Fork or Restore. Joining removes g immediately but keeps its continuation behind a barrier until both arms complete. An occurrence is enabled when minimal in some pomset of $\llbracket T \rrbracket$, consistent with sequence exposure by observable completion. Occurrence disjointness and unique context decomposition give every enabled x a unique exposed leaf $\text{leaf}_T(x) = b$. For $a \in \{\text{fresh}(x, d), \text{alias}(x, d)\}$, define the paired history residual

$$\text{HStep}((G, T), a) = (G \parallel (b, a), T/(x, a)), \quad b = \text{leaf}_T(x),$$

where $G \parallel (b, a)$ appends the progress record (b, a) , and $T/(x, a)$ residualizes the leaf and records any choice selection without storing a second cursor. Write $\text{HRes}((G, T), r)$ for iterating this paired residual over a resolved word r .

$\text{addr}(T)$ follows the same recursion over branch and group identifiers. It includes exposed groups before descending into their live arms and only the exposed sequence operand, excluding dormant losers and unexposed operands.

K maps checkpoint identifiers to immutable earlier continuation records. Write $C_k = \text{contract}(K(k))$ for the residual contract stored by checkpoint k . Σ_ζ is a versioned immutable registry of typed edit schemas. ρ contains the authenticated occurrence-to-gate/cell, handle, lineage, scope, canonical-invocation, and effect-digest data. For each live occurrence x , binding $\beta(x) = (j, \eta)$ names its gate and the domain's current epoch, while χ is the ordered append-only trace of durably *resolved* logical occurrences, including fresh and alias bindings, and ν is the history-view version. Because χ records authorization and causal progress rather than physical completion, the corresponding external result may still await remote settlement or return.

$\text{Save}(k, b)$ is an explicit checkpoint transition, not a seventh execution edit, and requires fresh k and addressable b . A trusted transaction snapshots the current pair (C_b, χ_b) by value and appends $(\omega, b, C_b, \chi_b, \rho|_{X_{C_b}}, \zeta)$ and its authenticated fingerprint to K . The checkpoint check requires $C_b = \Gamma_b/\text{raw}(\chi_b)$, and the transaction advances ν without changing the frontier, receipts, bindings, epochs, or authority. Both machines take this τ -transition, which makes a concurrent installation stale because installation authenticates H . Restore accepts only records created by this rule.

2) Well-Formedness:

Definition 1 (Well-formed history). *H is well formed when the following groups hold.*

1) *Structure.* G is acyclic, and continuation and group identifiers are globally unique across the recorded DAG and live frontier. Every addressable identifier has a unique context

decomposition, and every head of T is a live maximal node. Every checkpoint names an immutable node coordinate and a value-snapshotted residual/cursor pair derived from G .

- 2) Identity and progress. Occurrence and gate identifiers are unique, while occurrences mapped to the same cell agree on protected-call issuer, scope, canonical invocation, digest, and lineage. Resolved occurrences are downward closed in their pomset, consistent with every choice status, and absent from the residual frontier.
- 3) Live frontier. $\text{dom}(\beta) = X_{[T]}$, and every live occurrence x has exactly one authenticated binding $\beta(x) = (j_x, \eta)$ to the domain's single current epoch. Every live leaf $b:\mathcal{C}$ is exactly the indexed residual of its immutable registered base Γ_b by the raw projection of its resolved cursor χ_b , rather than a caller-pruned subfamily. Every constructor in T is defined, so every subtree interpretation is completion-observable.
- 4) Schema and domain. Every edge and introduced contract is justified by one immutable schema at version ζ . Every current, checkpoint, or schema occurrence, cell, gate, lineage, and scope belongs to ω . Cells, receipt ledgers, and policies of distinct domains are disjoint, and an edit never crosses a domain.

The durable kernel additionally requires that the one current policy-domain epoch is active.

3) *Ledger and Security Core:* Outside H , ledger state stores policy version κ , a regular prefix-closed authority policy $\mathcal{A} \subseteq U^*$ over label alphabet $\mathcal{U}$, monotone receipt ledger Δ , and ordered outbox out . Receipt creation is append-only. Settlement advances receipt phases and fills durable result futures with stable results. Each receipt binds its cell, immutable canonical invocation, creator occurrence, and authority label. Each record in χ points to its own fresh receipt or, for alias, an earlier receipt, making retry and alias-result lookup functions of (χ, Δ) . $\text{lab}(\Delta)$ is the authority word. Restore may reconstruct workspace payload but never restores Δ .

A security core $(\kappa, H, \Delta, \text{out}, \mathcal{A})$ is well formed when H is well formed, $\text{lab}(\Delta) \in \mathcal{A}$, each cell has at most one fresh receipt, each fresh record names its unique receipt, each alias record names an earlier receipt, the receipt ledger, outbox, and policy are scoped to ω , and the outbox preserves fresh order. This joint invariant is assumed below.

C. Adversary and trusted boundary

The adversary controls Agent output, edit requests, tool arguments, copied workspace, retries, stale authorization tokens, and scheduling, including races between calls and edits. It cannot forge registries, the checkpoint directed acyclic graph (DAG), effect-identity certificates, policy, or receipt log. The trusted computing base (TCB) includes the policy authority and protected-call authority. Within one policy domain, it completely mediates every protected use and provides a linearizable, crash-stable transaction for every modeled state change and visible edit answer. A recovered labeled transition system (LTS) state shows each transaction entirely uncommitted or committed according to whether its linearization point

occurred. No transaction is torn, and volatile compiler caches lie outside the model. Before the checker takes a source snapshot, the protected-call authority may instantiate finitely many pre-authorized schema templates. For tool request m , it computes normalized request $m^\circ = \text{canon}_\kappa(m)$, reuses only an effect identity certified for m° or allocates a fresh one, and signs $(x, j, d, \text{scope}, \text{digest}(m^\circ), \text{lineage})$ into the authenticated registry. The resulting finite registered call model R remains unchanged while that snapshot is checked. Unmatched calls fail closed. The platform supplies authenticated $\mathcal{W}, R$, edit schemas, normalization rules, policy, checkpoints, and one consistent snapshot of the receipt log and dispatch queue. From these inputs and an identifier-only Agent request, the controller derives the edited workflow, required outcomes, resolved calls, fixed point, monitor, certificate, monitor version, and authorization tokens. The Agent may supply or weaken none of these derived objects.

Definition 2 (Checked registration refinement). A boundary-finite typed Agent workflow $\mathcal{W}$ is a finite-state outcome-labeled LTS whose complete paths end at outcome terminals and whose τ -erased durable-site projections $\partial e = (o, s_1 \cdots s_n)$ form finitely many classes under equality $\equiv_\partial$. CheckReg decides this by rejecting any reachable and terminal-co-reachable strongly connected component containing a durable-site edge while permitting erased τ -cycles, then enumerates the finite classes. R stores a candidate registration map λ , and $\text{BRuns}(R)$ is the finite outcome-indexed language obtained by linearizing its registered root pomsets and labeling each occurrence by its protected Use. $\mathcal{W} \sqsubseteq_\lambda R$ means that λ induces a bijection from boundary classes to $\text{BRuns}(R)$ that preserves outcomes, incidence, and order. The finite checker also requires effect identities to be exactly the equivalence classes of normalized requests. The registration map must be total, unique, and free of raw instructions, and receipt evolution must commute with Resolve without dropping a required outcome, call site, or receipt.

Theorem 1 proves checker reflection and a source-boundary/registered-realization bijection. The theory neither infers natural-language contracts nor equates local commit with remote physical exactly-once execution, which additionally requires an idempotent or queryable sink.

D. Six Execution-Edit Rules

1) *Edit Certificates:* The six disjoint constructors of Edit_6 are exactly the typed operation signatures in table I. The set excludes Save, Root, Extend, protected use, and caller-supplied targets.

An untrusted request u supplies an edit kind, live object identifiers, and schema identifier σ , never a target contract, while Σ_ζ binds σ to that kind, exact source and checkpoint fingerprints, introduced suffix contracts, and a lineage certificate.

The edit certificate defined below makes the derivation checkable. For indexed source $\mathcal{C}$, schema-designated carried or cloned target region $\mathcal{D}$, and parent map $\mu : X_{\mathcal{D}} \rightarrow X_{\mathcal{C}}$, write $\mathcal{C} \preceq_\mu \mathcal{D}$ exactly when μ is a global bijection, some surjection

$\pi : O_D \rightarrow O_C$ satisfies the conditions below for every target outcome v , and μ preserves cell identity and lineage.

$$\begin{aligned} x \in X_{p_v} &\iff \mu(x) \in X_{p_{\pi(v)}}, \\ x \prec_v y &\iff \mu(x) \prec_{\pi(v)} \mu(y). \end{aligned}$$

Global bijection and occurrence–outcome membership equivalence prevent splitting a source occurrence shared across outcomes into outcome-specific target occurrences.

For candidate T' , let R_u index the disjoint carried or cloned target regions inside the replaced subtree. A rewrite certificate $\mathcal{P}_u = \{(C_r, \mathcal{D}_r, \mu_r, \pi_r)\}_{r \in R_u}$ contains the relation for each indexed region. A separate identity map checks that the exposed one-hole context preserves syntax, outcome indices, occurrences, cells, lineage, and order. Writing X_{ctx} for its occurrences and $X_{\text{suf}(u)}$ for schema-declared suffix occurrences, the checker verifies the following disjoint cover.

$$X_{[T']} = X_{\text{ctx}} \uplus \biguplus_{r \in R_u} X_{\mathcal{D}_r} \uplus X_{\text{suf}(u)}.$$

The row constructor and exposed-context syntax induce $\text{pr}_r^u : O_{[T']} \rightarrow O_{\mathcal{D}_r}$. For choice, this projection is defined only in the arm containing r , while product and sequence select the corresponding indexed component. Define

$$\begin{aligned} \text{RegionReq}_u &= \{(r, o) \mid r \in R_u, o \in O_{C_r}\}, \\ \text{Lift}_u(t, r, o) &\iff \exists v \in O_{\mathcal{D}_r}. \text{pr}_r^u(t) = v \wedge \pi_r(v) = o. \end{aligned}$$

Let O_{byp} be source-context outcomes whose alternatives bypass the hole, and let $\iota_{\text{ctx}}^u : O_{\text{byp}} \hookrightarrow O_{[T']}$ be the injection induced by context identity. Define the single required-outcome set and its bypass lift by

$$\begin{aligned} \text{Req}_u &= \text{RegionReq}_u \uplus (\{\text{ctx}\} \times O_{\text{byp}}), \\ \text{Lift}_u(t, (\text{ctx}, o)) &\iff t = \iota_{\text{ctx}}^u(o). \end{aligned}$$

For regional $a = (r, o) \in \text{RegionReq}_u$, $\text{Lift}_u(t, a)$ denotes the relation above. The checker also requires $\forall a \in \text{Req}_u. \exists t \in O_{[T']}$. $\text{Lift}_u(t, a)$, ensuring a full-target lift for every required source outcome, no outcome-specific splitting of shared occurrences, and no inherited occurrence mislabeled as a suffix.

The schema-verification phase checks the following judgment.

$$\Sigma_\zeta; H \vdash_{\text{edit}} u \Rightarrow (T', \mathcal{P}_u)$$

It holds exactly when every named current object belongs to $\text{addr}(T)$, every introduced object remains scoped to ω , the source fingerprint matches, the applicable $\mathcal{P}_u$, disjoint cover, and Lift_u requirements in table I hold, any removal is authorized, and $[T']$ is defined. Authorized removal is confined to a typed MergeSelect's losing arm or to a RestoreReplace's current continuation that is complete or explicitly certified for removal. The rule checks these recorded facts rather than Agent declarations. The removal certificate is an immutable record signed by the policy authority at version ζ . It binds (ω, σ) , the exact source fingerprint, the exact outcome indices and occurrence lineages that may be removed, and verifies that no other occurrence disappears. Its certified set must equal the typed losing arm for MergeSelect or the current continuation for RestoreReplace.

2) *Preserving required outcomes*: A row-tagged source outcome identifies one outcome that the edit must account for. Req_u contains those that must still be realized in the edited workflow. The occurrences of such an outcome are the calls needed to realize it.

Independently of T' , let $\text{SourceReq}(H, u)$ be all row-tagged source outcomes, $\text{Completed}_{H,u}$ those already completed at the current cursor, and $\text{Removed}_{H,u}$ the exact set whose removal the policy authority has signed. The checker enforces *required-outcome preservation*:

$$\text{SourceReq}(H, u) = \text{Req}_u \uplus \text{Completed}_{H,u} \uplus \text{Removed}_{H,u}.$$

Checked lifts preserve effect identity, lineage, and order. The installed edit retains completion evidence and records the authorized removal set, which the Agent cannot enlarge.

Let $\mathcal{E}[-]$ be a one-hole *exposed* frontier context, so its hole has the addressability property above. carry is the identity on continuation, outcome, and occurrence identifiers, pomset order, cells, lineage, base contracts, current branch cursors, and nested choice status. clone_r applies deterministic injective renamings to continuation, outcome, and occurrence identifiers, preserves pomset order and authenticated cell, scope, digest, and lineage data, and registers each cloned residual as a new base contract with an empty cursor. Schema suffix leaves likewise start with their registered base contract and an empty cursor. The schema-derived contracts $E_\sigma^L, E_\sigma^R, E_\sigma^J$ are suffixes, not replacements. Define

$$L_\sigma = \text{clone}_L(\mathcal{C}) \triangleright E_\sigma^L, \quad R_\sigma = \text{clone}_R(\mathcal{C}) \triangleright E_\sigma^R.$$

New identifiers and the target epoch η^+ are deterministically allocated from $(\omega, \zeta, \nu, u, \text{role})$. role identifies the edit row and operand position. Write $\text{Cl}_r(\mathcal{C})$ for the certified tuple $(\mathcal{C}, \text{clone}_r(\mathcal{C}), \mu_r, \pi_r)$, and $\text{Ca}(\mathcal{C})$ for $(\mathcal{C}, \text{carry}(\mathcal{C}), \mu, \pi)$. Each abbreviation asserts the corresponding $\preceq_\mu$ relation. For a frontier T_0 , $\text{Ca}(T_0)$ abbreviates $\text{Ca}([T_0])$ and has target contract $[\text{carry}(T_0)]$. Every b or g named below must lie in $\text{addr}(T)$.

In an open choice, the first fresh or alias atomically selects its arm and disables later occurrences from the other, so MergeSelect(g, w, σ) has no derivation after any durable loser progress, including receipt-silent progress. MergeJoin replaces the addressable $\parallel_g$ node with jbar , retiring g and preventing the same pair from being joined again.

The model uses domain-wide handoff, never fencing only a selected subset of a domain's gates, so every live gate in H belongs to $\eta^- = \eta$.

3) *Common Epoch Handoff*: The execution-edit judgment is the least relation generated by one row of table I inside a well-typed single-hole context $\mathcal{E}$, followed by the common update below; no other execution-edit derivation exists. Let e_u contain its typed edge and exactly the newly allocated clone, suffix, and group nodes prescribed by T' ; carried node identities remain unchanged. For every live target occurrence x , the trusted protected-call issuer deterministically mints j_x^+, h_x^+ , and ρ' preserves the row-certified cell, lineage, scope, digest,

TABLE I
SCHEMA-CHECKED AGENT EXECUTION EDITS. $E_\sigma^\bullet$ IS DERIVED FROM Σ_ζ , NEVER SUPPLIED AS A TARGET CONTRACT.

Operation u	Required current shape	Derived target shape	Checked obligation
ForkChoice(b, σ)	$\mathcal{E}[b:C]$	$\mathcal{E}[b_L:L_\sigma \oplus_g^{\text{open}} b_R:R_\sigma]$	$\{\text{Cl}_L(C), \text{Cl}_R(C)\}$
ForkParallel(b, σ)	$\mathcal{E}[b:C]$	$\mathcal{E}[b_L:L_\sigma \parallel_g b_R:R_\sigma]$	$\{\text{Cl}_L(C), \text{Cl}_R(C)\}$
RestoreReplace(b, k, σ)	$\mathcal{E}[b:C], k \in \text{dom } K$	$\mathcal{E}[b_k:\text{clone}_k(C_k)]$	$\{\text{Cl}_k(C_k)\}$; current retired
RestoreLive(b, k, σ)	$\mathcal{E}[b:C], k \in \text{dom } K$	$\mathcal{E}[b:\text{carry}(C) \parallel_g b_k:\text{clone}_k(C_k)]$	$\{\text{Ca}(C), \text{Cl}_k(C_k)\}$
MergeSelect(g, w, σ)	$\mathcal{E}[T_L \oplus_g^* T_R], s \in \{\text{open}, w\}$	$\mathcal{E}[\text{carry}(T_w); b_J:E_\sigma^J]$	$\{\text{Ca}(T_w)\}$; loser retired
MergeJoin(g, σ)	$\mathcal{E}[T_L \parallel_g T_R]$	$\mathcal{E}[\text{jbar}(\text{carry}(T_L), \text{carry}(T_R)); b_J:E_\sigma^J]$	$\{\text{Ca}(T_L), \text{Ca}(T_R)\}$

and historical records needed for Retry while refreshing live gate and handle fields. Define

$$\mathcal{H}_u^+ = \{(x, j_x^+, h_x^+) \mid x \in X[\llbracket T' \rrbracket]\},$$

$$\beta'(x) = (j_x^+, \eta^+).$$

The bundle is neither live nor caller-accessible before installation. The complete judgment is shown below.

$$\begin{aligned} \Sigma_\zeta; H \vdash u \Downarrow (H', \mathcal{C}_{H,u}, \eta^-, \eta^+), \\ H' = (\omega, G \cdot e_u, T', K, \Sigma_\zeta, \rho', \beta', \chi, \nu + 1, \eta^+), \\ \mathcal{C}_{H,u} = \llbracket T' \rrbracket, \quad \eta^- = \eta. \end{aligned} \quad (4)$$

Here $G \cdot e_u$ retains every source progress record and retired or carried node, and initializes each new clone or suffix node with its residual base and no progress records. Thus an edit preserves K, Σ_ζ, χ , appends one typed provenance edge and its target nodes, refreshes every live gate and handle, and moves the domain to η^+ . Table constructors determine new group modes, while carried nested modes remain unchanged. A request naming a nonexistent checkpoint, wrong group mode, nonmember winner, mismatched schema fingerprint, retirement without schema authorization, or undefined composition has no derivation. Independent domains step by asynchronous product, and no theorem allows fencing only a selected subset of one domain's gates.

Lemma 2 (Schema fidelity and handoff totality). *For well-formed H , the judgment in equation (4) is deterministic. If it derives H' , then H' is well formed. It preserves required outcomes: completion evidence remains recorded, and no pending source lineage disappears outside the exact set $\text{Removed}_{H,u}$. The identity-preserved context and family $\mathcal{P}_u$ preserve global occurrence sharing, cell, lineage, and causal order. For every $a \in \text{Req}_w$, some full target outcome t satisfies $\text{Lift}_u(t, a)$, including the identity lift of every bypassing context outcome. Every live target gate is bound to η^+ , has one authenticated handle in $\mathcal{H}_u^+$, and no target gate remains bound to η^- .*

Proof. Unique object and schema identifiers select one row and record, while context identity preserves everything outside the hole. The Fork rows certify both clones; RestoreReplace certifies the checkpoint clone and source retirement, while RestoreLive certifies the carried source and checkpoint clone. MergeSelect certifies the winner and typed loser, while MergeJoin certifies both carried regions before its barrier. Clone and

carry supply the global bijections and occurrence–outcome membership preservation; constructor induction defines pr_T^u , proves the cover, and supplies every Lift_u witness. The common poststate fixes all remaining fields and preserves or initializes branch cursors as specified above, while new identifiers, partial-constructor checks, and lemma 1 preserve uniqueness, acyclicity, and contract well-formedness. Finally, β' and $\mathcal{H}_u^+$ establish handoff totality at η^+ . $\square$

E. Registered extension

A registered extension adds new schemas and policy entries without changing the current workflow or its required outcomes. It is not a seventh execution edit. The Agent may name only an immutable record ξ binding the exact predecessor schema, registry, and policy roots to finite disjoint additions D_Σ, D_ρ and a successor policy $\mathcal{A}^+$. Both issuers authenticate ξ . Existing schema identifiers, occurrence–effect-identity and invocation rows, labels, scopes, lineages, receipts, and normalization rules retain their meanings. New identifiers are fresh, finite, domain-scoped, foreign-key closed, and completion-observable. The record contains no cursor, receipt, dispatch-queue entry, monitor version, return value, monitor, or installed certificate. The successor policy is regular and prefix closed and conservatively extends the old policy, $\mathcal{A} \subseteq \mathcal{A}^+$.

For fresh η^+ , refresh every current monitor entry and authorization token but leave the active workflow unchanged:

$$H_\xi^+ = (\omega, G, T, K, \Sigma_{\zeta^+}, \rho^+, \beta^+, \chi, \nu + 1, \eta^+),$$

$$\mathcal{C}_\xi = \llbracket T \rrbracket, \quad \text{SourceReq}(H, \xi) = \text{Req}_\xi = \mathcal{O}[\llbracket T \rrbracket],$$

$$\text{Completed}_{H,\xi} = \text{Removed}_{H,\xi} = \emptyset, \quad \text{Lift}_\xi(t, o) \iff t = o.$$

The checked judgment

$$(\kappa, H, \mathcal{A}) \vdash \text{Extend}(\xi) \Downarrow (\kappa^+, H_\xi^+, \mathcal{A}^+, \mathcal{C}_\xi, \eta, \eta^+)$$

preserves $G, T, K, \chi, \Delta, \text{out}$, every current outcome, occurrence, effect identity, lineage, and causal edge, and authorizes no removal. It appends one authenticated version record and then uses the same checker and atomic monitor installation as an execution edit.

IV. EXACT SAFETY CHECK FOR EXECUTION EDITS

The preceding section derives an edited workflow from the authenticated source record and requested edit. This section decides whether that workflow has a policy-safe implementation that preserves every still-possible required outcome. It first determines whether each copied call needs a new receipt or

reuses an existing one. It then computes the largest family of completed executions that remains safe after every partial execution. Finally, it presents the ideal atomic execution machine used to prove the installed runtime correct in section V.

Recall the authority-label alphabet $\mathcal{U}$ and regular prefix-closed policy $\mathcal{A} \subseteq \mathcal{U}^*$. For the authenticated receipt $\log \Delta$, $\text{lab}(\Delta) \in \mathcal{A}$ is its permission trace and D_Δ is the set of effect identities that already have receipts. Fix $\Theta = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$ and write $\text{src}(\Theta) = (\kappa, H, \Delta, \text{out}, \mathcal{A})$. When defined, the tagged root, execution-edit, or registered-extension derivation is unique. Write $\text{Derive}(\Theta) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. Root derivation additionally requires that $\text{CheckReg}(\mathcal{W}, R)$ accept. An execution edit has $(\kappa_u, \mathcal{A}_u) = (\kappa, \mathcal{A})$ and uses the unique execution-edit judgment. $\text{Extend}(\xi)$ uses the authenticated judgment in section III. Let ρ_u be the authenticated target registry in H_u , let $O_{H,u} = O_{\mathcal{C}_u}$, and use $H' = H_u$ and $\rho' = \rho_u$ where earlier notation is convenient. All languages below depend on Θ , the schema registry, ρ_u , and target policy $\mathcal{A}_u$, but not on the dispatch queue out. The executable checker nevertheless binds out into the authenticated source snapshot used for installation. We write only H, u where the remaining parameters are fixed.

A. fresh and alias resolution

Resolution never erases an occurrence retained by the derived contract. Starting with $D_0 = D_\Delta$, the deterministic transducer $\text{Resolve}_\Delta^{\rho_u}$ maps a raw word $w = x_1 \cdots x_n$ to a resolved word $z_1 \cdots z_n$. For $d_i = q_{\text{cell}}^{\rho_u}(x_i)$,

$$z_i = \begin{cases} \text{fresh}(x_i, d_i), & d_i \notin D_{i-1}, \quad D_i = D_{i-1} \cup \{d_i\}; \\ \text{alias}(x_i, d_i), & d_i \in D_{i-1}, \quad D_i = D_{i-1}. \end{cases} \quad (5)$$

The permission projection erases alias symbols and maps each fresh symbol to its effect identity's permission label.

$$\text{auth}(\text{fresh}(x, d)) = \ell_{\rho_u}(d), \quad \text{auth}(\text{alias}(x, d)) = \epsilon.$$

For resolved z , $\text{raw}(z)$ drops the fresh/alias mode and effect identity but preserves the ordered occurrence identifiers. A Retry repeats the same invocation and adds no occurrence. Restored occurrences sharing an effect identity remain distinct and may resolve differently.

Lemma 3 (Resolution). *Resolve $_\Delta^{\rho_u}$ is deterministic, length preserving, and prefix compatible. Its fresh projection contains each effect identity at most once outside D_Δ , while its occurrence projection is exactly the input word. Moreover, if $\text{Resolve}_\Delta^{\rho_u}(rv) = zv'$, then $z = \text{Resolve}_\Delta^{\rho_u}(r)$ and $v' = \text{Resolve}_\Delta^{\rho_u}(v)$, where Δ_z extends Δ by the fresh events of z .*

Proof. Induction on input length shows that the transducer emits one prefix-determined output symbol per input symbol and updates D_i only for fresh. Set membership ensures one fresh emission per effect identity. Both cases retain x_i . Splitting after r gives the streaming equation. $\square$

Thus a retry stutters in the resolved causal and authority histories, although its successful API reply is observable as $\text{Return}(t, [v]_\cong)$. A copied occurrence that resolves as alias

stutters only in the authority history because it advances causal history without changing the authority word.

B. Safe Completion After Every Prefix

1) *Largest family safe after every prefix:* For each derived outcome $o \in O_{H,u}$, define all of its completed executions and the subset allowed by policy.

$$\begin{aligned} R_o(H, u) &= \{\text{Resolve}_\Delta^{\rho_u}(w) \mid w \in \text{Lin}(p_o)\}, \\ M_o(H, u) &= \{z \in R_o(H, u) \mid \text{lab}(\Delta)\text{auth}(z) \in \mathcal{A}_u\}. \end{aligned} \quad (6)$$

Checking each outcome separately is insufficient. After a partial execution shared by several outcomes, every outcome still compatible with that partial execution must retain a policy-safe completion. The operator below removes exactly the completed executions that violate this condition. For a resolved prefix z , let

$$\text{Compat}(z) = \{o \mid \text{raw}(z) \preceq_{\text{pref}} w \text{ for some } w \in \text{Lin}(p_o)\}.$$

Put

$$\hat{B}_0 = \biguplus_{o \in O_{H,u}} \{o\} \times M_o.$$

For $\hat{B} \subseteq \hat{B}_0$, define the monotone operator

$$\hat{\Phi}(\hat{B}) = \left\{ (o, b) \in \hat{B}_0 \mid \begin{array}{l} \forall z \preceq_{\text{pref}} b. \forall v \in \text{Compat}(z). \\ \exists (v, b') \in \hat{B}. z \preceq_{\text{pref}} b' \end{array} \right\}. \quad (7)$$

Set $\hat{B}_{i+1} = \hat{\Phi}(\hat{B}_i)$. Since $\hat{B}_1 \subseteq \hat{B}_0$, monotonicity yields a finite descending chain, whose limit is

$$\hat{B}_{H,u}^\dagger = \nu \hat{B}. \hat{\Phi}(\hat{B}) = \bigcap_{i \geq 0} \hat{B}_i,$$

$$B_{H,u}^\dagger = \pi_2 \hat{B}_{H,u}^\dagger, \quad W_{H,u}^\dagger = \text{Pref}(B_{H,u}^\dagger).$$

Runtime uses $(W^\dagger, B^\dagger)$, while the checker and seal retain indexed witness $\hat{B}^\dagger$.

Definition 3 (Safe execution edit). *If $\text{Derive}(\Theta)$ is undefined, then $\text{SafeEdit}(\Theta)$ is false. Otherwise,*

$$\text{SafeEdit}(\Theta) \iff \hat{B}_{H,u}^\dagger \neq \emptyset. \quad (8)$$

$\text{SafeEdit}(H, u)$ fixes Θ and both registries. At $z = \epsilon$, nonemptiness covers every required outcome. The existential quantifier chooses a call order without assuming fairness, and the universal condition keeps every compatible outcome completable after every prefix. Relation Preserve requires every source outcome that is neither completed nor authorized for removal to have a checked lift into $\hat{B}$; definition 6 gives its full definition.

Definition 4 (Indexed realization). *A generated language and indexed marked family $(L, \hat{B})$ realize (H, u) when all conditions below hold.*

- 1) $\hat{B} \neq \emptyset$;
- 2) source and target fidelity: $\text{Preserve}(H, u, \hat{B})$, $\hat{B} \subseteq \biguplus_{o \in O_{H,u}} \{o\} \times R_o(H, u)$, and $L \subseteq \text{Pref}(\bigcup_{o \in O_{H,u}} R_o(H, u))$;
- 3) policy safety: $\text{lab}(\Delta)\text{auth}(z) \in \mathcal{A}_u$ for every $z \in L$;

- 4) completion nonblocking: $L = \text{Pref}(\pi_2 \hat{B})$; and
 5) persistent outcome coverage: for every $z \in L$ and $o \in \text{Compat}(z)$, some $(o, b) \in \hat{B}$ extends z .

Lemma 4 (Largest safe realization). *A realization exists iff $\text{SafeEdit}(H, u)$. When the edit is safe, $(W_{H,u}^\dagger, \hat{B}_{H,u}^\dagger)$ is a realization, and every realization $(L, \hat{B})$ satisfies $\hat{B} \subseteq \hat{B}_{H,u}^\dagger$ and $L \subseteq W_{H,u}^\dagger$.*

Proof. Fidelity, completion nonblocking, and safety place any realization's indexed marked family inside $\hat{B}_0$. Persistent coverage makes it post-fixed, so monotonicity and finite Knaster-Tarski place it inside $\hat{B}^\dagger$, and projected prefixes give $L \subseteq W^\dagger$. Conversely, $\hat{B}^\dagger = \hat{\Phi}(\hat{B}^\dagger)$ supplies persistent coverage. Together with checked lifts, this coverage establishes Preserve at ϵ . The inclusion $\hat{B}^\dagger \subseteq \hat{B}_0$ supplies target fidelity and safe marked words, while prefix closure of $\mathcal{A}_u$ supplies prefix safety. Finally, $W^\dagger = \text{Pref}(\pi_2 \hat{B}^\dagger)$ supplies nonblocking. Hence the fixed point is a realization exactly when nonempty. $\square$

$\text{GReal}(\Theta; L, \hat{B})$ declaratively denotes a realization containing every other one componentwise, and its unique witness is $\text{Spec}(\Theta)$. Lemma 4 proves that this witness exists exactly when $\hat{B}_{H,u}^\dagger \neq \emptyset$ and then equals $(W_{H,u}^\dagger, \hat{B}_{H,u}^\dagger)$.

The following shared-prefix instance shows why derived conditional markings matter even when every M_o is nonempty. Let singleton contracts X, Y, Z contain x_o, y_o, z_o , and let the target registry map them to distinct previously unreceipted cells d_x, d_y, d_z labeled x, y, z , respectively. Write $\bar{x} = \text{fresh}(x_o, d_x)$, $\bar{y} = \text{fresh}(y_o, d_y)$, and $\bar{z} = \text{fresh}(z_o, d_z)$.

2) *Why Individual Outcome Safety Is Insufficient:* For $\mathcal{C} = X \otimes (Y \oplus Z)$ and $\mathcal{A} = \text{Pref}\{yx, xz\}$, the $X \otimes Y$ and $X \otimes Z$ outcomes initially have safe resolved completions $\bar{y}\bar{x}$ and $\bar{x}\bar{z}$, with authority projections yx and xz . Yet prefix $\bar{x}$ from $\bar{x}\bar{z}$ remains contract-compatible with $X \otimes Y$ but leaves no safe continuation. The first fixed-point iteration removes $\bar{x}\bar{z}$, and the next removes $\bar{y}\bar{x}$ after root coverage is lost. Thus the edit is rejected, whereas the independent $\forall o. M_o \neq \emptyset$ test admits it.

Proposition 1 (Outcome-indexed nonblocking correspondence). *For nonempty $\hat{B} \subseteq \hat{B}_0$, let $G_{\hat{B}}$ be the prefix trie of $L_{\hat{B}} = \text{Pref}(\pi_2 \hat{B})$. Mark state z by α_o exactly when $o \in \text{Compat}(z)$, and mark state b by ω_o exactly when $(o, b) \in \hat{B}$. Then*

$$\begin{aligned} \hat{B} &\subseteq \hat{\Phi}(\hat{B}) \\ \iff \forall o \in O_{H,u}. G_{\hat{B}} \text{ is } (\alpha_o, \omega_o)\text{-nonblocking.} \end{aligned}$$

Consequently, if $\hat{B}^\dagger$ is nonempty, it is the greatest nonempty completion subfamily of $\hat{B}_0$ satisfying these conditional nonblocking obligations. If it is empty, no nonempty subfamily satisfies them. Ordinary marker nonblocking of the unaugmented projected language is strictly weaker.

Proof. From reachable prefix z , an ω_o -marked state is reachable exactly when some $(o, b) \in \hat{B}$ extends z . Quantifying over exactly the states marked α_o is therefore the post-fixed-point condition. Greatestness follows from the greatest-fixed-point

construction. For strictness, the preceding instance has the marker-nonblocking projected pair $(\text{Pref}\{\bar{y}\bar{x}, \bar{x}\bar{z}\}, \{\bar{y}\bar{x}, \bar{x}\bar{z}\})$, but its indexed fixed point is empty. $\square$

Proposition 1 gives the exact generalized-nonblocking interface [15], [16]: after Agent history derives α_o, ω_o from the edit, retirement, cells, receipts, and cut, that backend computes the same greatest family. Complete mediation makes fresh/alias controllable. Deployments with uncontrollable resolved events need the usual supremal-controllable refinement. Our characterization additionally constructs the greatest Agent monitor or impossibility certificate, carries it through atomic installation, and proves that neither a caller target nor a target-only trace can recover the derivation.

Each removed (o_{rem}, b) records its rank i , a prefix $z \preceq b$, and uncovered $o_{\text{miss}} \in \text{Compat}(z)$. Induction gives every post-fixed $P \subseteq \hat{B}_i$, so an empty final family excludes every realization, and installation recomputes the chain, causes, and survivor witnesses.

3) *Rejection Certificates and Residual Closure:* To preserve the safety result after each permitted prefix, fix $H, u, \mathcal{A}_u$ after deriving $\mathcal{C}$, and write $W^\dagger(\mathcal{C}, \Delta)$ and $\hat{B}^\dagger(\mathcal{C}, \Delta)$ with those parameters suppressed.

Lemma 5 (Residual coherence). *If $z \in W^\dagger(\mathcal{C}, \Delta)$, then $\text{Resolve}_{\Delta}^{\rho'}(\text{raw}(z)) = z$, $\text{lab}(\Delta_z) = \text{lab}(\Delta)\text{auth}(z)$, and*

$$\begin{aligned} W^\dagger(\mathcal{C}, \Delta)/z &= W^\dagger(\mathcal{C}/\text{raw}(z), \Delta_z), \\ \hat{B}^\dagger(\mathcal{C}, \Delta)/z &= \hat{B}^\dagger(\mathcal{C}/\text{raw}(z), \Delta_z), \end{aligned}$$

where $\hat{B}/z = \{(o, v) \mid (o, zv) \in \hat{B}\}$.

Proof. Because z prefixes a projected member of $\hat{B}^\dagger$, raw recoverability and streaming in lemma 3 give the recoverability and receipt-ledger equalities. The indexed residual removes exactly that raw prefix, retains every compatible outcome identity, and gives

$$\text{Compat}_{\mathcal{C}/\text{raw}(z)}(s) = \text{Compat}_{\mathcal{C}}(zs).$$

Together with authority concatenation, streaming resolution therefore yields the key equality

$$\hat{B}_0(\mathcal{C}, \Delta)/z = \hat{B}_0(\mathcal{C}/\text{raw}(z), \Delta_z),$$

with the same retained outcome identities, policy, and target registry. Thus $\hat{B}^\dagger/z$ is post-fixed for the residual operator and lies within its greatest fixed point Y . Conversely, streaming and residual policy safety put $zY = \{(o, zy) \mid (o, y) \in Y\}$ inside the original $\hat{B}_0$. The union $\hat{B}^\dagger \cup zY$ is post-fixed because prefixes $t \preceq z$ use existing witnesses, while prefixes $t = zs$ use witnesses in Y . Greatestness gives $zY \subseteq \hat{B}^\dagger$, hence $Y = \hat{B}^\dagger/z$. Finally, $\text{Pref}(B)/z = \text{Pref}(B/z)$ yields the generated-language equality. $\square$

Thus every accepted prefix retains a safe completion for every residual outcome index still contract-compatible with that prefix.

4) Live Extension:

Lemma 6 (Monotone live extension). *For an authenticated $\text{Extend}(\xi)$, the current residual indexed realization is a post-fixed family for the extension instance and is therefore contained in $\widehat{B}_{H, \text{Extend}(\xi)}^\dagger \neq \emptyset$. Consequently, a valid live extension cannot retire or strand a currently promised outcome.*

The proof is in section A-A.

C. The ideal atomic history machine

The ideal labeled transition system (LTS) retains abstract gate and epoch identities for protected calls and history edges but omits the compiler, monitor representation, phase protocols, seals, preloading, and handoff microsteps. An ideal state is $I = (\kappa, H, \Delta, \text{out}, \mathcal{A}, c, r)$, where $c = (H_c, u, L^*, \widehat{B}^*)$. $H.\eta$ and $H.\beta$ are its current abstract epoch and live bindings.

a) *History edge:* For $\Theta_I(u) = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$, the rule $I \xrightarrow{\text{Edge}(u, [\mathcal{H}_u^\dagger]_\cong)} (\kappa_u, H_u, \Delta, \text{out}, \mathcal{A}_u, (H, u, L^*, \widehat{B}^*), \epsilon)$ exists exactly when $\text{Derive}(\Theta_I(u)) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$ and $\text{Spec}(\Theta_I(u)) = (L^*, \widehat{B}^*)$. H_u records the epoch handoff and target gates, while Spec is independent of Admit , Compile , and install . Undefined derivation yields the state-preserving $\text{Invalid}(u, [\gamma, C_{\text{str}}]_\cong)$ with canonical failed-check transcript, whereas a derivation with no GReal witness yields the state-preserving $\text{Reject}(u, [\gamma, C_{\text{fp}}]_\cong)$ with canonical ranked empty-chain. Together with Edge , these current- $\gamma(\Theta_I(u))$ verdicts are mutually exclusive and exhaustive.

b) *Protected occurrence:* For submitted invocation m , an enabled current-frontier occurrence x passes the ideal guard exactly when $H.\beta(x) = (j, H.\eta)$, its authenticated handle binds j and cell d , and $\text{canon}_\kappa(m) = \text{inv}_\rho(d)$. Current receipts then determine $a \in \{\text{fresh}(x, d), \text{alias}(x, d)\}$. Exactly when $ra \in L^*$, one atomic transition applies $\text{HStep}((G, T), a)$, appends a to the global resolved-occurrence trace χ , advances ν , removes x from $H.\beta$, and sets $r := ra$. fresh also appends to Δ an immutable receipt for d labeled $\ell(d)$ and enqueues $(d, \text{canon}_\kappa(m))$. alias changes neither field.

An invalid occurrence, gate, handle, or invocation, a disabled branch, or $ra \notin L^*$ fails the guard and returns $\text{deny}(x)$ without changing state. alias binds the restored occurrence to its cell's durable result future, advancing causal history without inventing a result not yet produced by the original invocation.

c) *Retry, release, and crash:* Save takes the trusted checkpoint transition in section III as a τ -step. An authenticated same-invocation retry follows its χ -to-receipt link and emits $\text{Return}(t, [v]_\cong)$ for the same stable result future without changing history, whereas an unauthenticated or argument-changing lookup is denied. Dispatch releases exactly the stored invocation in the oldest prepared outbox item and marks that item released, making the release trace a prefix of fresh preparation order. Settlement emits $\text{Settle}(d, [v]_\cong)$, monotonically advances a receipt's phase, and fills its durable result future with the stable result, changing no history or authority field. Crash discards nondurable computation without changing LTS state. We make no claim about completion

order inside an external network after release. The ideal LTS is the least relation generated by these history-edge verdicts, protected-use/denial, Save , $\text{retry/retrieval/delivery}$, Dispatch , settlement , and recovered-crash rules. No other transition, including compiler computation, exists.

By lemma 4, every reachable ideal-machine prefix retains a policy-safe completion until the next explicit history edge changes the contract. The next sections implement this machine without consulting the full completion set at every call.

V. MONITOR COMPILATION AND DURABLE INSTALLATION

The safety check produces either an impossibility certificate or the greatest safe family of completed executions. The durable kernel turns a nonempty family into enforcement in three steps: (1) compile its prefix closure into a canonical monitor, (2) bind the monitor to the authenticated source cut, and (3) atomically install it under a fresh epoch. The rules below make racing protected calls occur entirely before or after this installation boundary.

A. Occurrence-sensitive compilation

The greatest realization has the finite, canonically installable projected pair $(W, B) = (W_{H,u}^\dagger, B_{H,u}^\dagger)$. For $L/z = \{v \mid zv \in L\}$, use residual states $q_z = (W^\dagger/z, B^\dagger/z)$.

$$\begin{aligned} Q_{W,B} &= \{q_z \mid z \in W\}, \\ q^0 &= q_\epsilon = (W^\dagger, B^\dagger), \\ q_z &\xrightarrow{a} q_{za} \iff za \in W. \end{aligned} \quad (9)$$

Call this automaton $\text{Can}(W, B)$. Every state is reachable and is marked exactly when $\epsilon \in B^\dagger/z$. Each transition symbol a contains occurrence x , cell d , and its fresh or alias mode, so a caller cannot use another occurrence's gate.

Lemma 7 (Exact compiled realization). *The generated and marked languages of the program in equation (9) are exactly $(W_{H,u}^\dagger, B_{H,u}^\dagger)$. The program is the smallest deterministic marked partial automaton for this pair up to state renaming.*

Proof. Induction on z shows that the automaton reaches $(W^\dagger/z, B^\dagger/z)$, enables a iff $za \in W^\dagger$, and marks the reached state iff $z \in B^\dagger$. Distinct states differ on a generated or marked suffix and must remain distinct. The automaton is the paired-language derivative construction [19]. $\square$

For $\Theta = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$, let $\gamma(\Theta) = \text{Seal}_{\text{cut}}(\text{enc}(\text{request}, \omega, \zeta, \kappa, H, \Delta, \text{out}, \mathcal{A}, u))$ bind a verdict to its request cut.

$$\begin{aligned} \text{Result}(\Theta) &::= \text{Invalid}([\gamma, C_{\text{str}}]_\cong) \mid \text{Reject}([\gamma, C_{\text{fp}}]_\cong) \\ &\quad \mid \text{Installable}(\delta, W^\dagger, \widehat{B}^\dagger, Z). \end{aligned}$$

$\text{Compile}(\Theta)$ returns Invalid with the canonical failed-check transcript if derivation is undefined, Reject with the complete ranked empty-chain if $\widehat{B}^\dagger = \emptyset$, and otherwise Installable with the unique δ , coverage witnesses, $\text{prog}_Z = \text{Can}(W^\dagger, B^\dagger)$, and

$$Z = (\omega, \zeta, \kappa_Z, \text{cut}_Z, u, H_{\text{prog}}, \eta^-, \eta^+),$$

where $\kappa_Z = \kappa$, $H_c = H$ is the authenticated pre-installation history, and the history judgment mints η^+ . Canonical encoding order makes all three finite semantic results unique and equivariant; the bounded wrapper may instead return $\text{ResourceLimit}(b, n)$, which installs nothing and proves neither safety nor impossibility.

Let enc be canonical and length-delimited, with κ authenticating $\mathcal{A}$. The symbolic LTS uses domain-separated injective authenticated constructors $\text{Seal}_{\text{prog}}$ and Seal_{cut} , so equal seals have equal payloads. Define

$$H_{\text{prog}} = \text{Seal}_{\text{prog}}(\text{enc}(\text{prog}_Z, \widehat{B}_{H,u}^\dagger)).$$

The cut seal is

$$\text{cut}_Z = \text{Seal}_{\text{cut}}(\text{enc}(\omega, \zeta, \kappa, H, \Delta, \text{out}, u, \mathcal{C}_{H,u}, H_{\text{prog}}, \eta^-, \eta^+)). \quad (10)$$

Because H contains χ, ν, ρ, β , the cut seal binds logical progress, the history view, registry state, and gate/epoch bindings, and it binds the receipt ledger and outbox directly. Search and inactive preloading remain untrusted because the installation checker re-derives the schema judgment and complete indexed fixed point, reconstructs $(W^\dagger, B^\dagger)$, verifies both seals, and checks

$$\text{prog}_Z \cong \text{Can}(W^\dagger, B^\dagger),$$

as a full automaton isomorphism preserving the initial state, marking, and every labeled transition. A byte implementation may use signatures or collision-resistant digests over enc , yielding computational refinement up to forgery or collision, while exact bisimulation uses the symbolic constructors.

B. Durable kernel

The compiled state for one policy domain is

$$S = (\kappa, H, \Delta, \text{out}, \mathcal{A}, \text{phase}, \text{bind}, \text{prog}, \text{q}, \text{cert}),$$

where phase is a finite partial map with $\text{phase}(\eta) = \perp$ for an unallocated epoch and values inactive, active, or closed otherwise. Reserve $u = \text{Root}(R, n)$ for bootstrap only. The root rule is enabled only when $\text{CheckReg}(\mathcal{W}, R)$ accepts a finite-state boundary-finite typed Agent workflow $\mathcal{W}$ and finite registered slice R . The deterministic judgment $\text{Reg}(R, n) \vdash \text{Root} \Downarrow (H_R^+, \mathcal{C}_R, \perp, \eta_R)$ constructs $\text{src}_R^- = (\kappa, H_R^-, \epsilon, \epsilon, \mathcal{A})$ from registered slice R and private namespace n , with empty progress, checkpoints, bindings, and epoch, sets Req_{Root} to R 's root outcomes, and defines $\text{Lift}_{\text{Root}}(t, o) \iff t = o$. The checked record $\text{cert}(\eta) = c = (\text{src}_c, H_c^+, u, W, \widehat{B}, Z)$ contains source cut $\text{src}_c = (\kappa, H_c, \Delta_c, \text{out}_c, \mathcal{A})$, pre-progress target H_c^+ , and ideal-cut projection $\text{icut}(c) = (H_c, u, W, \widehat{B})$. Its tagged origin is the checked root judgment exactly when $u = \text{Root}(R, n)$ and $\text{src}_c = \text{src}_R^-$; otherwise it is the checked derivation of either one of the six execution edits or an authenticated live extension. Its indexed family is proof-only ghost state, while $W^\dagger, B^\dagger$, and the seal are checked enforcement data; $\text{Gen}(P, q)$ and $\text{Mark}(P, q)$ denote P 's generated and marked languages from q .

Definition 5 (Agent security-state invariant). For $c = (\text{src}_c, H_c^+, u, W, \widehat{B}, Z)$ and resolved r , $\text{AgentSec}(S; c, r)$ holds exactly when the following clauses hold.

- 1) Core and schema coherence. The current security core is well formed, $c = \text{cert}(\eta)$, and the tagged origin derives $(H_c^+, \mathcal{C}, \eta^-, \eta)$ by its corresponding checked root, execution-edit, or live-extension judgment; each $a \in \text{Req}_u$ has a checked full-target Lift_u witness.
- 2) Promise coherence. $\widehat{B} = \widehat{B}_{H_c, u}^\dagger \neq \emptyset$, $W = \text{Pref}(\pi_2 \widehat{B})$, and the stored certificate passed $\text{Verify}_Z(W, \pi_2 \widehat{B}, \widehat{B})$ at its source cut.
- 3) Execution coherence. $(H.G, H.T) = \text{HRes}((H_c^+.G, H_c^+.T), r)$, $H.\chi = H_c^+.\chi r$, $\text{lab}(\Delta) = \text{lab}(\Delta_c)\text{auth}(r)$, and each occurrence in r appears once in χ . Each fresh has its unique receipt and ordered outbox preparation, each alias names an earlier receipt, receipt phases and releases advance only monotonically, and K differs from $H_c^+.K$ only by authenticated Save extensions.
- 4) Monitor coherence. For $B = \pi_2 \widehat{B}$, checked installation gives $\text{prog}(\eta) \cong \text{Can}(W, B)$ and makes $\text{q}(\eta)$ correspond to canonical derivative $q_r = (W/r, B/r)$.
- 5) Epoch coherence. Only η is active. Each live occurrence has exactly one authenticated gate and handle bound to η . Inactive epochs expose no handle. Closed epochs never reactivate. Each target epoch deterministically minted at the current cut for a registered request is unallocated or inactive and unbound.

The full system starts from one empty genesis, accepts a checked finite registration containing only contracts, schemas, policy, and authenticated identity rows, and then performs root installation to construct empty receipt/cursor/outbox state, the first epoch, its certificate, and its monitor. Registration cannot supply runtime fields, while every later cut is produced by an execution-edit derivation or an authenticated additive live extension.

a) *Atomic protected use*: For submitted invocation m , set $m^\circ = \text{canon}_\kappa(m)$; for x bound to gate j in epoch η , $\text{VerifyHandle}(h, \rho(x), j, d, m^\circ)$ verifies the issuer, scope, gate, cell, lineage, and signed digest and requires $m^\circ = \text{inv}_\rho(d)$. Set $a = \text{fresh}(x, d)$ if $d \notin D_\Delta$, otherwise $a = \text{alias}(x, d)$, and let $\text{Use}_x(m) = \text{Use}(x, j, h, m)$. Its sole durable transition is the rule below.

$$\frac{\begin{array}{l} x \text{ enabled in } T \quad \text{bind}(x) = (j, \eta) \\ \eta = H.\text{epoch} \quad \text{phase}(\eta) = \text{active} \\ \text{VerifyHandle}(h, \rho(x), j, d, m^\circ) \quad \text{q}(\eta) \xrightarrow{\text{prog}(\eta)} q' \end{array}}{S \xrightarrow{\text{Use}_x(m)/a} S'} \quad (11)$$

The unique structured residual $T/(x, a)$ removes x from its leaf contract, selects the containing arm beneath an open choice, and preserves every frontier constructor. The paired update $\text{HStep}((G, T), a)$ also appends a to the unique live branch

cursor. $\text{Advance}(S, x, a, q')$ changes exactly the fields below.

$$\begin{aligned} (H.G, H.T) &:= \text{HStep}((G, T), a), & H.\chi &:= \chi a, \\ H.\nu &:= \nu + 1, & H.\beta &:= \beta \setminus \{x\}, \\ \text{bind} &:= \text{bind} \setminus \{x\}, & q(\eta) &:= q'. \end{aligned}$$

In addition, fresh appends the immutable labeled receipt and ordered outbox item $(d, \text{inv}_\rho(d))$, never an unchecked caller buffer, while alias records the cell's durable-result-future link without changing Δ , and all other fields remain unchanged.

An already-resolved x is a Retry exactly when $\text{VerifyRetry}(h, x, j, d, m^\circ, \chi, \Delta)$ authenticates the same canonical invocation through the archived χ -to-receipt link. If the linked future contains stable value v under its authenticated reply token t , success is the state-preserving observable transition $S \xrightarrow{\text{Return}(t, [v]_\cong)} S$. For unresolved x , equation (11) applies exactly when all premises hold, while a missing transition, bad handle, unbound occurrence, disabled branch, or closed epoch returns $\text{deny}(x)$ without changing state. Dispatch releases exactly the stored canonical invocation in the oldest outbox item, settlement preserves AgentSec and its authority projection, and crash is a state self-loop. Here Dispatch is the durable local outbox release, not evidence of exactly-once execution by a remote service.

1) *Verdict Verification and Atomic Installation*: At its linearization point, the kernel ignores caller-supplied target and epoch data, sets $\Theta_S(u) = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$, and re-runs $\text{Derive}(\Theta_S(u))$ from the current state. VerifyInv accepts exactly a current γ , an undefined derivation, and a canonical failed-check transcript. In contrast, VerifyRej accepts exactly a current γ , a defined derivation, an empty fixed point, and a canonical elimination chain. Both read-only transactions preserve state and emit the corresponding Invalid or Reject label. For the positive case, write $\text{Derive}(\Theta_S(u)) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. $\text{ReCut}(S, u, Z)$ is undefined without this root, execution-edit, or live-extension derivation and otherwise recomputes the right-hand side of equation (10). $\text{Verify}_Z(W, B, \hat{B})$ holds exactly when the trusted checker (1) recomputes $\hat{B}_0$ and every fixed-point iteration, (2) verifies all ranked removal causes and surviving coverage witnesses, (3) checks the stabilized $\hat{B} \neq \emptyset$, (4) reconstructs $B = \pi_2 \hat{B}$ and $W = \text{Pref}(B)$, (5) establishes $\text{prog}_Z \cong \text{Can}(W, B)$, and (6) verifies the program seal. Installation requires

$$\begin{aligned} \kappa &= \kappa_Z, & \text{ReCut}(S, u, Z) &= \text{cut}_Z, \\ \text{Verify}_Z(W_{H,u}^\dagger, B_{H,u}^\dagger, \hat{B}_{H,u}^\dagger), & & & \\ H.\text{epoch} &= \eta^-, & \text{phase}(\eta^+) &\in \{\perp, \text{inactive}\}. \end{aligned} \quad (12)$$

$\text{Ready}(S, u, Y)$ holds when $Y = \text{Installable}(\delta, W^\dagger, \hat{B}^\dagger, Z)$, the current derivation is δ , and all state-dependent premises of equation (12) hold in S . If these premises hold, one domain transaction verifies that every target binding in H_u names a new gate owned by η^+ and every replacement triple in $\mathcal{H}_u^+$

passes the handle check against ρ_u and its registered cell. Let $c^+ = (\text{src}(\Theta_S(u)), H_u, u, W^\dagger, \hat{B}^\dagger, Z)$. It then applies

$$\begin{aligned} \kappa &:= \kappa_u, & \mathcal{A} &:= \mathcal{A}_u, \\ H &:= H_u, & \text{phase}(\eta^-) &:= \text{closed}, \\ \text{phase}(\eta^+) &:= \text{active}, & \text{bind}|_{X_{[H_u.T]}} &:= H_u.\beta, \\ \text{prog}(\eta^+) &:= \text{prog}_Z, & q(\eta^+) &:= q^0, \\ \text{cert}(\eta^+) &:= c^+. \end{aligned}$$

The transaction preserves unrelated inactive or closed records, exposes $\mathcal{H}_u^+$ only after η^+ becomes active, and lets recovery retrieve a lost reply from the durable target registry. Old handles for unresolved occurrences fail the refreshed registry/gate check, whereas same-invocation Retry follows its archived χ -to-receipt link. Success is observed as $\text{Edge}(u, [\mathcal{H}_u^+]_\cong)$, retaining the schema-determined reply bundle modulo private-name renaming. A stale or malformed precomputed candidate emits no verdict, is discarded as an internal stutter, and must be recomputed against the current request cut before any visible response.

2) *Auxiliary Kernel Transitions*: $\text{Preload}(\eta, Z)$ is a τ -step that may allocate an unallocated epoch or change an inactive, unbound epoch, and it leaves the epoch inactive, while a stale or malformed candidate attempt is a state-preserving τ -step. Save takes the trusted checkpoint transition in section III. Authenticated retry, bundle retrieval, and result delivery emit $\text{Return}(t, [v]_\cong)$ without changing the security core, while settlement emits $\text{Settle}(d, [v]_\cong)$ and monotonically fills the corresponding durable result future. Crash is a security-state self-loop. Dispatch(d) is enabled only for the outbox head and marks exactly that entry released. The compiled LTS is the least relation generated by equation (11) and the visible Invalid/Reject/Edge, denial, preload, Save, retry/retrieval/delivery, Dispatch, settlement, stale-candidate, and crash rules above. Pure Compile is meta-level.

Lemma 8 (History-cut linearization). *In every reachable well-formed compiled state, a visible edit verdict and a protected use in the same policy domain have a unique order and decision cut. If the use commits first, that seal cannot install. If Install commits first, the old use cannot resolve fresh or alias. Steps in independent domains commute by product semantics.*

Proof. Prior fresh/alias invalidates every old cut seal, while prior installation closes the old epoch; disjoint-domain steps commute, and the persisted target and certificate restore the invariant. $\square$

C. Decidability and trust

Proposition 2 proves an output-sensitive $O(D + n\Lambda + |\Gamma| + L \log(2 + L))$ -time, $O(D + \Lambda + |\Gamma|)$ -space bound; exponential growth occurs only in explicitly emitted linearizations and obligations. The TCB is the registries, ledger, atomic rules, policy/call issuers, certificate verifier, and complete mediation; independently rechecked search, minimization, and preloading are untrusted.

VI. EXACT EDIT-SAFETY CHARACTERIZATION

The main theorem connects the preceding layers: authenticated derivation, the exact safety check, monitor compilation, and durable installation. It also characterizes which history information an exact implementation must retain and proves closure across finite executions. The theorem has one central equivalence and three supporting guarantees. Clause 2 equates realizability, successful compilation, and atomic installation. Clauses 1 and 3 justify the checked boundary and the necessity of its state, while clauses 4 and 5 lift the result to finite histories and the durable API.

Fix a derivation witness $\delta = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. An independently defined *installed atomic realization* is a pair $(M, \hat{B}_M)$ whose generated language realizes (H, u) according to definition 4 and whose marked language is $\pi_2 \hat{B}_M$. The witness's exact-cut transaction must atomically install $(\kappa_u, H_u, \mathcal{A}_u, M)$ at η^+ , close η^- , bind the target gates, and publish replacement handles.

a) *Relational evidence*: We call S the compiled runtime state, AgentSec its invariant, and $\text{Sec}(S; c, r)$ the four-factor information view used by the lower bound. At a checked cut, $\text{Sec}(S; c, r) = \mathbf{P} \bowtie \mathbf{I} \bowtie \mathbf{E} \bowtie \mathbf{C}$ joins promised topology/schema, occurrence-cell identity, effect progress/receipts, and epoch/cut coordinates. $\mathcal{Q}_{\text{sec}}$ contains typed decide/install queries, and $\text{Exact}(\tau)$ means that a decoder reproduces Ans from $\tau(\text{Sec})$ for every such query. Section A-C gives the full field dependencies, projections, query semantics, and private-name orbits. The lower bound concerns information rather than a particular layout.

b) *Durable Agent-API observation*: $\mathcal{O}_{\text{api}}$ exposes verdicts and history edges bound to a specific cut, protected-use outcomes, dispatch, settlement, and authenticated returns. It quotients private names but preserves public bytes. obs_{api} hides only internal computation, preload, Save, stale attempts, and recovered crash. $\mathcal{R}$ equates ideal and compiled states when AgentSec , cut and prefix, epoch, live bindings, and visible fields agree. Section A-G gives its typed maps, reply linkage, and good initial pairs.

Theorem 1 (Exact Edit-Safety). *Under section III's boundary, clause 1 holds for every finite-state $\mathcal{W}$ and finite R . Clauses 2–5 hold for every $u \in \text{Edit}_6 \uplus \{\text{Extend}(\xi)\}$ at every finite reachable state satisfying AgentSec . Registration acceptance entails boundary-finiteness.*

- 1) **Registration reflection**. $\text{CheckReg}(\mathcal{W}, R)$ accepts iff stored λ witnesses $\mathcal{W} \sqsubseteq_\lambda R$. Source executions modulo $\equiv_\delta$ are then order-isomorphic to $\text{BRuns}(R)$, preserving outcomes, order, cells, fresh/alias, the safety decision, and the greatest monitor. Missing promises, inconsistent cells, or unmediated durable edges reject before bootstrap.
- 2) **Exact compilation**. For $\Theta = \Theta_S(u)$, both machines share one trichotomy bound to the same decision cut. An undefined derivation gives a verified, state-preserving Invalid . A unique δ with $\hat{B}_\delta^+ = \emptyset$ gives a verified, state-preserving Reject and no installed atomic realization. Otherwise, let $Y = \text{Compile}(\Theta)$ and c_δ^+ be the checked record. In this

positive case, the following conditions are equivalent: (1) $\hat{B}_\delta^+ \neq \emptyset$, (2) an installed atomic realization exists, (3) $\text{Ready}(S, u, Y)$ holds, and (4) an $\text{Edge}(u, [\mathcal{H}_u^+]_\cong)$ successor satisfies $\text{AgentSec}(S^+; c_\delta^+, \epsilon)$. The greatest witness and canonical monitor satisfy SPC , while the edge retains $\text{Done}_{H,u}$'s evidence and records $\text{Dec}_{H,u}$. The three verdict classes are mutually exclusive and exhaustive at the same decision cut.

- 3) **Factor tightness**. The full join is sufficient, and every proper factor projection is insufficient:

$$\text{Exact}(\pi_J) \iff J = \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}.$$

Every exact abstraction distinguishes table II; incidence erasures and caller-proposed target-only τ_{tgt} remain inexact, while generalized nonblocking becomes exact after trusted target/marking derivation.

- 4) **Sequence closure**. Every enabled fully parameterized event induces the corresponding partial derivative on the private-name equivalence classes of Sec and preserves AgentSec . Iteration covers every finite prefix of edits, live extension, protected use, ledger, installation, and crash, with finite cuts but no uniform history-length bound.
- 5) **Durable Agent-API realization**. $\mathcal{R}$ is a divergence-insensitive weak bisimulation under obs_{api} , so every good pair satisfies $S_0 \approx_{\mathcal{O}_{\text{api}}}^{\text{di}} I_0$. The observation includes verdicts tied to the authenticated history cut, bundles, and results, but excludes time, fairness, and post-Dispatch networking. Disjoint domains compose asynchronously.

Proof sketch. Schema conservation and finite-checker reflection prove Clause 1, fixed-point reasoning proves Clause 2, opposite-answer pairs prove Clause 3, and event induction together with durable simulation proves Clauses 4 and 5. The appendix supplies the witnesses and durable simulation. $\square$

VII. EXECUTABLE VALIDATION

a) *Proof coverage*: The executable validation checks the finite decision core and its correspondence to the formal development. Five Lean modules pass `--trust=0`, with their theorem mapping listed in section A-B. The factor bounds, pomset lifting, and bisimulation remain mathematical proofs rather than mechanized Lean results.

b) *Executable checks*: The explicit compiler also remains practical for the bounded instances exercised by the artifact. Across 81 decision-kernel tests, five-process medians on a 16-vCPU AMD EPYC 7R12 with CPython 3.12 are 53.47 ms for a 128-outcome safety check, 5.51 ms for rejection, and 50.35 ms for 720 unordered completions. Instances beyond the configured cap fail closed.

VIII. RELATED WORK

a) *Recovery and rollback*: Output-commit recovery delays effects until state commits and reconstructs a fixed computation after failure. It does not admit topology-changing Agent edits under indexed outcomes and shared authority [20], [21].

b) Supervisory control and synthesis: Generalized non-blocking and live synthesis consume supplied plants and markings or old specifications and executions. Our checker instead derives the obligations of all six edits and the necessary-and-sufficient condition for an exact safety decision from authenticated Agent history before installation [15], [16], [17].

c) Agent recovery mechanisms: ACRFence, DART, Atomix, and Rebound fence post-Restore replay, choose preserving checkpoints, settle after frontiers, or make rollback atomic and auditable. These mechanisms take logs, scopes, classes, frontiers, or policies as inputs rather than deriving the exact safety condition from an authenticated execution record [22], [14], [12], [23].

IX. CONCLUSION

Agents that Fork, Restore, or Merge an execution require a safety decision that accounts for both logical control state and irreversible external progress. Our checker derives this decision from a checked workflow, authenticated history, durable receipts, and the requested edit. Its greatest fixed point yields the most permissive safe monitor when the edit is realizable, while an empty fixed point yields a finite impossibility certificate. The characterization theorem connects this safety result to atomic installation, identifies the history information required for exact decisions, and preserves the characterization across finite sequences of edits, protected calls, crashes, and additive extensions. The Lean modules and executable tests validate the finite checked core and its explicit compiler.

APPENDIX A
SUPPLEMENTARY PROOF APPENDIX

This appendix is clearly marked as supplementary material and does not enlarge the model or claims of the main paper. It supplies the proof objects used by theorem 1 for one finite registered protected-call slice and one atomic policy-domain epoch at a time. Independent domains retain the product interpretation stated in the paper.

A. The Declarative Specification and Its Computed Witness

Fix a well-formed safety-check instance $\Theta = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$ for which the history judgment derives $\delta = (H^+, \mathcal{C}, \eta^-, \eta^+)$. All sets in this subsection are indexed sets, so two equal resolved words belonging to different outcomes remain different elements.

Definition 6 (Source-promise coverage).

$$\text{SPC}(H, u, \hat{B}) \iff \forall a \in \text{Req}_u. \exists t \in O_{H,u}. b. \text{Lift}_u(t, a) \wedge (t, b) \in \hat{B}.$$

Here $\text{Dec}_{H,u}$ contains the typed losing-arm outcomes for MergeSelect or the current-role outcomes of a pending certified RestoreReplace. $\text{Done}_{H,u}$ contains the completed current role of RestoreReplace. Both sets are empty for every other row and for live extension. Thus $\text{Req}_u = \text{SrcProm}(H, u) \setminus (\text{Done}_{H,u} \uplus \text{Dec}_{H,u})$, so SPC covers exactly the promises neither already discharged nor explicitly declassified.

Let $\mathfrak{R}_\Theta$ be the finite set of indexed realizations from definition 4, ordered componentwise by

$$(L_1, \hat{B}_1) \sqsubseteq (L_2, \hat{B}_2) \iff L_1 \subseteq L_2 \wedge \hat{B}_1 \subseteq \hat{B}_2.$$

Lemma 9 (Declarative–algorithmic bridge). *The declarative predicate $\text{GReal}(\Theta; L, \hat{B})$ has a witness if and only if $\hat{B}_{H,u}^\dagger \neq \emptyset$. When a witness exists, it is unique and equals $(W_{H,u}^\dagger, \hat{B}_{H,u}^\dagger)$. This equality is a theorem relating two independently stated objects, rather than the definition of the ideal machine.*

Proof. Let $(L, \hat{B}) \in \mathfrak{R}_\Theta$. Source and target fidelity and policy safety give $\hat{B} \subseteq \hat{B}_0$. For every $(o, b) \in \hat{B}$, every prefix $z \preceq_{\text{pref}} b$ belongs to $L = \text{Pref}(\pi_2 \hat{B})$. Persistent outcome coverage therefore supplies, for every $v \in \text{Compat}(z)$, a pair $(v, b') \in \hat{B}$ extending z . Thus $\hat{B} \subseteq \hat{\Phi}(\hat{B})$. By monotonicity on the finite lattice $2^{\hat{B}_0}$, every post-fixed set is contained in the greatest fixed point, so $\hat{B} \subseteq \hat{B}^\dagger$. Taking projected prefixes yields $L \subseteq W^\dagger$.

Conversely, suppose $\hat{B}^\dagger \neq \emptyset$. The fixed-point equality supplies persistent coverage and, at ϵ , composes with checked lifts to establish SPC; inclusion in $\hat{B}_0$ supplies target fidelity and safe completed words, prefix closure of $\mathcal{A}$ supplies safety for every generated prefix, and $W^\dagger = \text{Pref}(\pi_2 \hat{B}^\dagger)$ supplies completion nonblocking. Hence $(W^\dagger, \hat{B}^\dagger) \in \mathfrak{R}_\Theta$, and the preceding containment makes it greatest. If two greatest elements existed, each would contain the other componentwise, so they would be equal. If $\hat{B}^\dagger = \emptyset$, the first half places every hypothetical realization inside the empty set, contradicting its required nonemptiness. $\square$

Corollary 1 (Exact compilation boundary). *The ideal history edge is enabled by the declarative $\text{Spec}(\Theta)$ exactly when the checked compiler returns a nonempty fixed point. On that edge, the ideal language L^* equals the generated language of $\text{Can}(W^\dagger, B^\dagger)$.*

Proof. The first statement is lemma 9. The second combines that lemma with lemma 7. $\square$

Proposition 2 (Output-sensitive compilation). *Fix the unit-cost symbolic-checker model with constant-time finite-map lookup and policy-automaton transitions. Let*

$$\begin{aligned} D &= |\Theta|_{\text{chk}}, & n &= \max_o |X_{p_o}|, \\ \Lambda &= \sum_o \sum_{w \in \text{Lin}(p_o)} (|w| + 1), & K &= |\hat{B}_0|, \\ L &= \sum_{(o,b) \in \hat{B}_0} (|b| + 1). \end{aligned}$$

$$\Gamma = \{((o, b), z, v) \mid (o, b) \in \hat{B}_0, z \preceq b, v \in \text{Compat}(z)\}.$$

Here D is the authenticated structural input size, Λ is indexed linearization volume, and Γ is the materialized prefix–outcome obligation relation. The semantic compiler runs in $O(D + n\Lambda + |\Gamma| + L \log(2 + L))$ time and $O(D + \Lambda + |\Gamma|)$ space, including its emitted witnesses. Moreover, $K \leq \sum_o |\text{Lin}(p_o)|$, $|\Gamma| \leq K(n + 1)|O_{C_u}|$, and the number of nonempty strict-removal ranks is at most K .

Proof. A backtracking topological-order enumerator emits all indexed linearizations in $O(n\Lambda)$ time, while resolution, policy simulation, and construction of raw and resolved prefix tries are linear in emitted volume. For every support key (z, v) , maintain the number of live pairs (v, b') extending z , initially enqueue candidates having a zero-count obligation, and remove candidates in synchronous batches. When removal makes a support count zero, enqueue exactly the candidates incident to that obligation for the next batch. Each candidate and each incidence in Γ is processed at most once, and the batches are exactly $\hat{B}_i \setminus \hat{B}_{i+1}$. Store one rank and cause per removed candidate rather than copied sets, and construct $\text{Can}(W, B)$ by bottom-up sorting of finite-trie derivative signatures in $O(L \log(2 + L))$ time. $\square$

Proof of lemma 6. An authenticated extension is identity-on-frontier, preserves cell resolution and all indexed outcomes, and satisfies $\mathcal{A} \subseteq \mathcal{A}^+$, so every old safe completion and coverage witness remains valid for the successor operator. The current residual family is therefore post-fixed, and greatestness gives its inclusion in a nonempty successor fixed point. $\square$

B. Lean Theorem Map

RegistrationRefinement.
checkRegistration_iff,
RegistrationRefinement.compiled_cell_eq
iff_canonical_eq, RegistrationRefinement.
compiled_mediation_exhaustive, the three
receipt theorems, and RegistrationRefinement.

compiled_admission_preserved cover registration reflection and its link to the safety decision. FiniteCore.compiler_eq_semantic and the greatest-realization theorems cover the fixed point, while HistoryStructure.deriveEdit_iff covers all six structural derivations. CompilationBridge.compiler_admit_exists_secure_install constructs a valid installed cut from an accepted edit, and its converse declarations recover acceptance from any valid install candidate. OperationalSemantics.kernelTrace_preserves_agentSec, OperationalSemantics.live_extension_preserves_agentSec, OperationalSemantics.fresh_before_live_extension_stale, OperationalSemantics.alias_before_live_extension_stale, and OperationalSemantics.live_extension_before_old_use_denied cover installed finite-prefix closure and the extension races. The audit module imports these declarations and prints only the project-allowed axioms propext, Classical.choice, and Quot.sound.

C. Normalized Evidence and Safe Projection

This subsection asks which parts of the authenticated security state are necessary to reproduce every safety-check and installation answer. We first normalize private identifiers so that representation choices cannot distinguish two states, then remove one information factor at a time and construct queries whose correct answers differ.

We make the normalization implicit in Sec explicit. Let Q_{sec} pair every valid $V = \text{Sec}(S; c, r)$ with a well-sorted $q ::= \text{Decide}(u) \mid \text{TryInstall}(u, Y)$, permitting stale names. Ans returns $\text{Compile}(\Theta_V(u))$ for Decide, and for TryInstall returns the successor bundle exactly when $\text{Ready}_V(u, Y)$, otherwise NoCommit. An equivariant τ is exact iff some D_τ maps every $\langle \tau(V), q \rangle_{\cong}$ to $[\text{Ans}(V, q)]_{\cong}$. Let $\mathcal{K}$ be the finite typed key universe at a checked cut, containing tagged outcome, occurrence, cell, receipt, checkpoint, group, domain, epoch, policy-version, and schema-version keys. Public keys are authority labels and identifiers declared external by the interface, while every other key may be consistently renamed. For a typed query q , $\text{supp}(q)$ is defined recursively over its full syntax, including all identifiers nested inside a supplied symbolic certificate, monitor, or cut seal. Let $\mathcal{F}$ be the finite set of typed field paths and assign every semantic base field a unique owner

$$\text{own} : \mathcal{F}_{\text{base}} \longrightarrow \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}.$$

P owns history constructor tags, live/checkpoint/target contract nodes, order and outcome membership, schema kind and source fingerprints, lift relations, and retirement declarations. **I** owns occurrence–cell incidence, cell–canonical–invocation/label/scope rows, occurrence lineage, and logical gate and handle slots. **E** owns the global resolved trace, live and checkpoint cursors, receipt–cell/creator and retry paths, result futures, ordered outbox phase rows, and residual policy. **C** owns the domain, policy/schema/view versions, private

namespace, epoch allocation and phase, active epoch and gate ownership, certificate origin and coordinates, and checkpoint cut coordinates. These four lists are disjoint and exhaustive for semantic base fields. Certificate bytes, seals, fingerprints, digests, derived authority words, and foreign-key paths are derived fields rather than additional semantic facts.

Write $f \rightsquigarrow g$ when the value or well-typed interpretation of derived field g depends immediately on field f . The dependency graph is acyclic because canonical encodings are constructed from base relations in a fixed order. Every foreign-key path depends on its target. Authenticated gate and handle bytes depend on their **I** slot and **C** epoch and owner. Receipt reconstruction and the authority word depend on the **E** receipt and cursor rows together with their **I** cells and labels. Target certificates, cut seals, and digests depend on every encoded base field in their payload. A normalized full view V is a compatible finite assignment to all base fields together with the unique values of all derivable fields. Compatibility requires equal values on shared typed keys and the well-formedness conditions of definition 1. The natural join $\mathbf{P} \bowtie \mathbf{I} \bowtie \mathbf{E} \bowtie \mathbf{C}$ is the compatible union of these uniquely owned base fields followed by deterministic recomputation of derived fields. It is undefined precisely when shared typed keys disagree or a required foreign-key target is absent.

For $J \subseteq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, start with base fields owned by J and the public sort declarations needed to type the retained records. Repeatedly delete any foreign-key path whose target has been deleted and any derived field having a deleted transitive dependency. The unique fixed point of this deletion is $\text{Ret}(J)$, and $\pi_J(V)$ is V restricted to $\text{Ret}(J)$. This definition prevents an erased fact from surviving inside a seal, digest, archived receipt-only registry path, or certificate payload.

The incidence-erasure projection $\pi_{\neg oc}$ starts from the full view, deletes the occurrence–cell relation, and applies the same dependency deletion while retaining the occurrence and cell marginals. The projection $\pi_{\neg rc}$ analogously deletes receipt–cell incidence and every path that reconstructs it, including receipt–creator, cursor–receipt, and archived creator–cell paths, while retaining the receipt, creator, occurrence, and cell marginals. State views are compared modulo a sort-preserving bijection on private keys that fixes all public keys. For a common literal query q , write $V \cong_q V'$ when the witnessing bijection additionally fixes $\text{supp}(q)$ pointwise; equivalently, $\langle V, q \rangle_{\cong} = \langle V', q \rangle_{\cong}$.

Lemma 10 (Projection is well defined). *Natural join, dependency deletion, and private renaming commute. Consequently, $[\pi_J(V)]_{\cong}$, $[\pi_{\neg oc}(V)]_{\cong}$, and $[\pi_{\neg rc}(V)]_{\cong}$ do not depend on a representative or on a certificate serialization.*

Proof. A sort-preserving bijection preserves key equality, foreign-key reachability, ownership, and the dependency relation. It therefore maps each deletion round to the corresponding deletion round in the renamed view. Induction on the finite acyclic dependency graph proves that the retained fixed points correspond. Canonical recomposition is equivariant because it is a typed constructor over exactly those retained fields. $\square$

Erased	Full-view pair	Exact answers
$u_R(C) := \text{RestoreReplace}(b, k, \sigma)$ with $C_k = C$.		
P	$q_P = \text{MergeSelect}(g, L, \sigma);$ $T^0 = x \oplus_g^{\text{open}} y, T^1 = x \parallel_g y;$ otherwise equal.	V^0 : realize; V^1 : Invalid (wrong mode).
I	$q_I = \text{RestoreLive}(b_L, k, \sigma_I)$ after safe empty-root/ForkChoice/Save; $\Delta = \epsilon, \mathcal{A} = \text{Pref}\{a\};$ $q_{\text{cell}}^0(x) = q_{\text{cell}}^0(y) = d_0;$ $q_{\text{cell}}^1(x) = d_0 \neq d_1 = q_{\text{cell}}^1(y);$ $\ell(d_0) = \ell(d_1) = a.$	$X \parallel Y_k$: one fresh admit; two fresh reject <i>aa</i> .
E	$q_E = \text{TryInstall}(u_R(x), Y_0); Y_0$ is compiled at V^0 after safe empty-root/ForkChoice/Save/fresh; $V^0 \xrightarrow{\text{Dispatch}(d)} V^1$, changing only the release phase.	V^0 : install; V^1 : NoCommit (stale cut).
$\pi_{\neg rc}$	$q_{\neg rc} = u_R(x)$ after safe empty-root/ForkChoice/Save/fresh; $q_{\text{cell}}(x) = d, \mathcal{A} = \text{Pref}\{a\}$; same marginals $\{p\}, \{d, e\}; p \mapsto d$ in V^0 , $p \mapsto e$ in V^1 .	$\text{alias}(x_k, d)$: admit; $\text{fresh}(x_k, d)$: reject <i>aa</i> .
C	$q_C = \text{TryInstall}(u_0, Y_0)$, with full package Y_0 compiled at V^0 , is literal in cut. both; name supplies agree off the epoch sort, while live epoch, ownership, and dependent seals differ.	V^0 : install; V^1 : reject stale

TABLE II

FACTOR- AND INCIDENCE-ERASURE PAIRS WITH ISOMORPHIC RETAINED VIEWS AND OPPOSITE EXACT ANSWERS.

Lemma 11 (Representation-independent observational separation). *Let V_0, V_1 be finite valid full views and let q be a typed query. If $[\text{Ans}(V_0, q)]_{\cong} \neq [\text{Ans}(V_1, q)]_{\cong}$, then every exact abstraction τ satisfies $\tau(V_0) \not\cong_q \tau(V_1)$. The claim applies to arbitrary encodings and is not a minimal-table or unique-representation claim.*

Proof. Assume toward contradiction that $\tau(V_0) \cong_q \tau(V_1)$. The joint decoder inputs $\langle \tau(V_0), q \rangle_{\cong}$ and $\langle \tau(V_1), q \rangle_{\cong}$ then coincide. Exactness forces equal answer orbits, contradicting the premise. $\square$

D. Bootstrap States and Explicit Separation Pairs

Every separation proof follows the same recipe. Starting from a common checked genesis, we vary exactly one security-state factor, keep the visible projection and literal query fixed, and obtain opposite correct answers. The resulting pair proves that an exact abstraction cannot erase that factor.

We construct the finite lower-bound witnesses from one literal empty genesis S_0 . A registered slice R compiled from a boundary-finite typed Agent workflow $\mathcal{W}$ contains only finite contracts, schemas, policy, lowering data, and authenticated occurrence, cell, invocation, lineage, scope, and label rows. It contains no receipt, cursor, outbox, epoch, certificate, result future, monitor state, or installed cut. The pre-slice relation has a registration step $S_0 \xrightarrow{\text{register}(\mathcal{W}, R, n)} \text{Reg}(R, n)$ exactly when $\text{CheckReg}(\mathcal{W}, R)$ accepts and n is a fresh private namespace, followed by checked root installation. Root installation derives the initial cut and constructs empty receipt, cursor, and outbox state, generation zero, one active epoch, the canonical monitor, and authenticated live bindings.

Lemma 12 (Registration reflection). *For finite-state $\mathcal{W}$ and finite R , $\text{CheckReg}(\mathcal{W}, R)$ accepts iff the registered λ witnesses $\mathcal{W} \sqsubseteq_{\lambda} R$ in definition 2. On acceptance, the λ -lowerings of source executions modulo $\equiv_{\partial}$ correspond bijectively to $\text{BRuns}(R)$. The correspondence preserves indexed outcomes, causal order, canonical invocation quotienting, receipt evolution, and the safety-check answer.*

Proof. The productive-SCC test terminates because the state graph is finite. A productive SCC containing a durable-site edge can be pumped before an outcome, producing projected words of unbounded length. Conversely, if no productive SCC contains such an edge, every productive SCC is boundary-silent, and contracting them yields a DAG with a finite enumerable projected language. The checker compares that language with $\text{BRuns}(R)$ and directly decides outcome/site incidence, order, cell attestation, total unique lowering, raw-bypass exclusion, and prefix receipt commutation. Therefore it accepts exactly the semantic conjunction defining $\mathcal{W} \sqsubseteq_{\lambda} R$, rather than an oracle restatement. Boundary projection erases only internal steps, so matching projected paths induce the quotient-execution bijection. Canonical invocation equality and deterministic Resolve make receipt evolution commute with this bijection at every prefix. The realization families are therefore order-isomorphic, so the fixed-point operator preserves the safety decision, greatestness, and pullback of the monitor. $\square$

Lemma 13 (Natural bootstrap). *If $\text{CheckReg}(\mathcal{W}, R)$ accepts and R 's registered root contract and policy have a nonempty greatest realization, registration and checked root installation reach a finite state S_R with $\text{AgentSec}(S_R; c_R, \epsilon)$. The cut origin is the checked root derivation, while every later cut-changing transition records an execution-edit derivation or an authenticated live extension.*

Proof. Registration contributes only the checked R and n , and deterministic allocation constructs every runtime identifier from them. The root judgment and checked lifts establish core/schema coherence, and the recomputed nonempty fixed point establishes promise coherence. Empty runtime progress establishes execution coherence, the canonical program at q_{ϵ} establishes monitor coherence, and the atomic phase/binding assignments establish epoch coherence. Only initial installation creates a root origin, and every successful later installation stores its checked execution-edit or live-extension derivation. $\square$

The following finite pairs use private names except for each displayed query and authority labels. Each displayed state is reached from S_0 by the stated registration, root-installation, Save, protected-use, and edit steps; omitted fields are identical up to the stated private renaming and dependency deletion.

a) **P: structural topology:** Let X and Y be singleton contracts with distinct occurrences x and y , common cells, and a universal residual policy. From the same registered singleton root, checked initial installation followed by ForkChoice

reaches V_P^0 , while the corresponding ForkParallel trace reaches V_P^1 . The two views differ only in

$$V_P^0.T = b_L:X \oplus_g^{\text{open}} b_R:Y, \quad V_P^1.T = b_L:X \parallel_g b_R:Y.$$

Use the same registered MergeSelect schema and query $q_P = \text{MergeSelect}(g, L, \sigma)$. The choice view has a unique structural derivation and a nonempty one-step greatest realization, whereas the parallel view has no MergeSelect derivation because its group mode is wrong. Deleting **P** removes the constructor tag and every authenticator depending on it, leaving isomorphic retained views.

b) *Bootstrap convention for the incidence pairs*: Let **1** be the singleton-outcome contract whose unique pomset is empty. Its completion ϵ gives a nonempty greatest realization under $\mathcal{A} = \text{Pref}\{a\}$. Both pairs below install this safe root and then a checked ForkChoice. Each dormant schema row needed to equalize cell marginals is identical in both views and never becomes live.

c) **I**: *occurrence-cell incidence*: Let X, Y be singleton contracts with occurrences x, y , and let d_0, d_1 be unreceipted cells labeled a . Two checked registered slices have identical topology, schemas, occurrence marginals, and cell marginals but the following incidence:

	$q_{\text{cell}}(x)$	$q_{\text{cell}}(y)$
V_I^0	d_0	d_0
V_I^1	d_0	d_1

From the installed **1** root, the common Fork schema derives $b_L:X \oplus_g^{\text{open}} b_R:Y$. Each outcome redeems one cell, so both Forks install under $\text{Pref}\{a\}$. Save b_R before progress and issue the common query $q_I = \text{RestoreLive}(b_L, k, \sigma_I)$. Up to clone renaming, the target is $(X \otimes Y_k) \oplus Y$. In V_I^0 , the left outcome uses one fresh and one alias on d_0 , while the bypass outcome uses one fresh, so every outcome has authority word a . In V_I^1 , every left-outcome linearization uses fresh on both d_0, d_1 , so its word is $aa \notin \mathcal{A}$. The left outcome is compatible with ϵ , so the first $\hat{\Phi}$ round also removes the otherwise safe bypass completion and leaves the greatest fixed point empty. Erasing occurrence-cell incidence and its dependent rows makes the retained views isomorphic while preserving equal marginals.

d) **E**: *mutable effect progress*: Let C, X be singleton contracts with occurrences c, x , let $q_{\text{cell}}(c) = q_{\text{cell}}(x) = d$, and label d by a . From the safe **1** root, a checked Fork installs $b_L:C \oplus_g^{\text{open}} b_R:X$; save b_R , then use c to complete the left arm, create receipt p , and prepare its outbox item. At the resulting V_E^0 , compile $u_E = \text{RestoreReplace}(b_L, k, \sigma_E)$ to the full package Y_0 ; its clone of x aliases d , so Y_0 is installable under $\mathcal{A} = \text{Pref}\{a\}$. Let $V_E^0 \xrightarrow{\text{Dispatch}(d)} V_E^1$, which changes only **E**'s outbox-release phase and preserves **P, I, C** literally. For the common literal $q_E = \text{TryInstall}(u_E, Y_0)$, V_E^0 installs, whereas V_E^1 returns NoCommit because the cut seal binds the outbox phase.

e) $\pi_{\neg rc}$: *receipt-cell incidence*: For a separate pair, retain $q_{\text{cell}}(x) = d$ and cells d, e labeled a , but let the archived creator bind $p \mapsto d$ in $V_{\neg rc}^0$ and $p \mapsto e$ in $V_{\neg rc}^1$. Both states follow the same safe empty-root/ForkChoice/Save/fresh

shape above, have authority word a , and use the common query $q_{\neg rc} = \text{RestoreReplace}(b_L, k, \sigma_{\neg rc})$. The cloned x_k is $\text{alias}(x_k, d)$ in $V_{\neg rc}^0$, but $\text{fresh}(x_k, d)$ produces forbidden word aa in $V_{\neg rc}^1$. Here the only differing occurrence-cell row belongs to the archived receipt creator; $\pi_{\neg rc}$ explicitly deletes that reconstructing path together with receipt-cell incidence, leaving isomorphic retained views with equal receipt, occurrence, and cell marginals.

f) **C**: *history cut*: Let a bootstrap namespace be a family $n = (n_s)_{s \in \text{Sort}}$ of injective, per-sort fresh supplies. From the same literal empty genesis, register the same public slice R with supplies n^0, n^1 that agree on every non-epoch sort but mint distinct initial epochs $\eta_0 \neq \eta_1$, and perform checked root installation. The resulting V_C^0, V_C^1 have literally identical **P, I, E** base fields, while **C**'s active epoch, gate ownership, certificate coordinates, and dependent authenticators differ. Compile the same registered edit u_0 at V_C^0 to obtain $Y_0 = \text{Installable}(\delta_0, W_0, \hat{B}_0, Z_0)$, and submit the same literal query $q_C = \text{TryInstall}(u_0, Y_0)$ to both views. At V_C^0 , recomputation reproduces cut_{Z_0} , so installation succeeds. At V_C^1 , recomputation binds η_1 and its owner rather than the η_0 -coordinates sealed in Z_0 , so installation rejects the stale cut. Erasing **C** recursively erases epoch-dependent gate/handle rows, certificate coordinates, seals, and authenticators; the identity bijection on every retained sort then witnesses $V_C^0 \cong_{q_C} V_C^1$.

Proposition 3 (Explicit factor and incidence lower bounds). *Every pair above consists of finite states reached from the common empty genesis and satisfying AgentSec. Each pair has q -isomorphic indicated projections and opposite answer orbits for its common literal query. Therefore every proper factor projection, $\pi_{\neg oc}$, and $\pi_{\neg rc}$ is inexact. Every exact abstraction must distinguish all displayed pairs.*

Proof. Lemma 13 gives the common genesis and valid root installation, while the explicit traces above use only modeled Save, protected-use, and execution-edit transitions. For the incidence pairs, the empty root completes with ϵ , and each installed Fork arm has a one-fresh completion labeled a . Hence every displayed pre-query state is reachable under $\text{Pref}\{a\}$. Open choice enables the Save steps, and the installed monitors enable each stated left-arm fresh before that arm becomes a completed, still-addressable continuation. For the **E** pair, Dispatch preserves **P, I, C** but changes an outbox phase bound by Y_0 's cut seal, giving install versus NoCommit; the displayed schema and resolution calculations give the other opposite answers. Lemma 10 and the specified private-name bijections, each pointwise fixing the common query support, give q -relative projection equality. Any projection omitting at least one factor collapses the corresponding factor pair, while the two incidence erasures collapse their corresponding incidence pairs. Inexactness and the general information lower bound now follow from lemma 11. $\square$

Assigning the same caller-proposed target plant, markings, and epoch to the **P** pair makes τ_{tgt} equal while leaving the authenticated queries and opposite answers unchanged.

Thus target-only pre-derivation state is not exact, although generalized nonblocking is exact as a backend after trusted derivation supplies the plant, outcome identities, and conditional markings.

E. Event Derivatives and Invariant Preservation

Write $A_1, \dots, A_5$ for the five clauses of AgentSec in their stated order. The following matrix is exhaustive for durable transitions inside an installed registered protected-call slice. Initial registration occurs before the root safety check; later additive schema and policy growth is the authenticated live-extension history edge listed below.

Lemma 14 (Representative-independent event derivative). *The guarded update induces a partial function ∂ on joint state-event orbits $\langle V, e \rangle_{\cong}$. If $\text{AgentSec}(S; c, r)$ and $S \xrightarrow{e} S'$, the witness in table III satisfies $\text{AgentSec}(S'; c', r')$ and*

$$[\text{Sec}(S'; c', r')]_{\cong} = \partial(\langle \text{Sec}(S; c, r), e \rangle_{\cong}).$$

Proof. All rule guards use typed equality, membership, order, canonical constructors, and authenticated payload equality. These operations are equivariant under a simultaneous private-key renaming of the state and event payload. For every such renaming π , $G_e(V) \iff G_{\pi e}(\pi V)$ and $U_{\pi e}(\pi V) = \pi U_e(V)$. Thus equal pointed orbits have the same definedness and the same successor orbit. This proves representative independence and functionality.

For fresh and alias, lemmas 3, 5 and 7 establish the first two rows of the matrix. If x lies at b , HStep appends a to b 's cursor, while the same atomic protected-use update appends a to the global resolved-occurrence trace χ ; residual associativity gives

$$(\Gamma_b/\text{raw}(\chi_b))/x = \Gamma_b/\text{raw}(\chi_b a).$$

Thus the updated leaf and any later Save remain synchronized with G . For a successful history edge, lemmas 2 and 6 to 9 establish the third row for all six schema rows and authenticated live extension. Each remaining rule changes only the fields shown in the matrix, and its row checks every invariant clause whose fields change. Thus every successor has the displayed witness and normalized update. $\square$

Corollary 2 (Finite-prefix closure). *Starting from any valid bootstrap, every finite prefix of any sequence of the event classes in table III ends in a state satisfying AgentSec . The sequence length is unbounded, although every individual safety-check instance and event payload is finite. For a finite word $\bar{e}$, the iterated derivative is defined on $\langle V, \bar{e} \rangle_{\cong}$, where one renaming acts on the initial view and the whole word.*

Proof. The base case is lemma 13, and lemma 14 supplies both the invariant-preserving inductive step and equivariance under one renaming of the remaining event suffix. $\square$

F. Uncontrollable Events and Recovered-State Crashes

The resolved alphabet contains only commits mediated by the protected monitor. Agent request arrival, scheduling, settlement, delivery, and crash are environmental events, but

none appends a fresh or alias symbol. Request arrival and scheduler choice have no durable transition until a guarded rule commits. Settlement and delivery emit modeled API observations but leave the resolved prefix unchanged. Dispatch is observable only when the oldest prepared item is durably released, and it also leaves the resolved prefix unchanged. Therefore these uncontrollable events stutter with respect to the conditional-nonblocking plant state. A deployment in which an environmental action itself commits a resolved occurrence is outside this contract and must instead synthesize a supremal controllable sublanguage.

Lemma 15 (Uncontrollable stutter). *If $\text{AgentSec}(S; c, r)$, e is an uncontrollable modeled event, and $S \xrightarrow{e} S'$, then the resolved prefix and canonical monitor state are unchanged and $\text{AgentSec}(S'; c, r)$ holds.*

Proof. Request arrival and scheduling have no durable transition, while denial, retry, retrieval, delivery, and crash preserve the recovered security state. Dispatch and settlement change only monotone outbox, receipt-phase, or result-future fields permitted by execution coherence. None changes $H.T$, $H.\chi$, the authority projection of Δ , r , or $q(\eta)$, and the corresponding row of table III preserves every other invariant clause. $\square$

The crash statement uses the recovered-state interface assumed in section III, not a new storage protocol. Let D be a concrete durable image, let v be volatile state, and let t be at most one in-flight domain transaction. Linearizability and crash stability provide a recovery function satisfying

$$\text{Recover}(D, v, t) = \begin{cases} D, & t \text{ has not linearized,} \\ \text{commit}_t(D), & t \text{ has linearized.} \end{cases}$$

No recovered image contains a strict subset of one modeled atomic update. In particular, a fresh transaction cannot expose a receipt without its global-trace record, branch-local cursor record, and outbox preparation, and installation cannot activate a new epoch without closing the old epoch and installing all target bindings.

Lemma 16 (Recovered-state crash refinement). *If a concrete execution refines compiled state S immediately before a transaction linearizes, recovery refines S . If the transaction linearizes to a rule successor S' , recovery refines S' . At recovered-state LTS boundaries, crash is consequently a τ self-loop.*

Proof. The recovery equation selects the entire preimage or the entire committed postimage. The former has the same abstraction as S , while the latter has the unique guarded update described by the corresponding row of table III and therefore abstracts to S' . Once the LTS records the recovered image, crashing again changes only volatile state and hence becomes a self-loop. $\square$

G. Durable Agent-API Weak Bisimulation

We now prove the durable-realization clause of theorem 1 by exhausting the observable and silent cases. For $K =$

Event class	Durable update	Witness	Preservation argument
fresh use	Apply HStep to (G, T) ; append $\text{fresh}(x, d)$ to global χ ; advance ν, q ; append one receipt and canonical-invocation outbox item; remove live binding.	$(c, r\text{fresh}(x, d))$	A_1 : registry and schema unchanged. A_2 : residual coherence. A_3 : unique receipt and ordered preparation. A_4 : canonical derivative. A_5 : remove exactly x 's binding.
alias use	Apply HStep to (G, T) ; append $\text{alias}(x, d)$ to global χ ; advance ν, q ; add only the result-future link; remove live binding.	$(c, r\text{alias}(x, d))$	A_1 : authenticated cell retained. A_2 : residual coherence. A_3 : link names an earlier receipt and authority is unchanged. A_4 : canonical derivative. A_5 : remove exactly x 's binding.
Successful history edge, six edits or live extension	Install derived $(\kappa^+, H^+, \mathcal{A}^+)$, checked monitor and certificate; close η^- ; activate η^+ ; publish new bindings.	(c_δ^+, ϵ)	A_1 : schema fidelity and lifts. A_2 : declarative-algorithmic bridge and checker. A_3 : source receipts and outbox retained. A_4 : install at q_ϵ . A_5 : total domain handoff.
Verified Invalid or Reject	Emit the canonical current-cut certificate; no durable state mutation.	(c, r)	All clauses unchanged; γ binds the visible answer to this decision cut.
Save	Snapshot the current residual/cursor pair in an authenticated checkpoint; advance ν .	(c, r)	A_1 : immutable well-typed extension. A_2, A_4, A_5 : unchanged. A_3 : exactly the permitted K extension.
Preload	Allocate or replace content only in an inactive unbound epoch.	(c, r)	A_1 – A_4 : unchanged. A_5 : inactive and unbound remain true; no handle is exposed.
Retry, retrieval, delivery	Emit $\text{Return}(t, [v]_\cong)$ for an authenticated durable value; no security-state mutation.	(c, r)	All clauses unchanged; retry additionally checks the same canonical invocation and receipt link.
Denial	Return $\text{deny}(x)$; no state mutation.	(c, r)	All clauses unchanged.
Dispatch(d)	Mark exactly the oldest prepared outbox item released.	(c, r)	A_3 : release order remains a prefix of preparation order. Other clauses unchanged.
Settlement	Emit $\text{Settle}(d, [v]_\cong)$, advance one receipt phase, and fill its stable result future.	(c, r)	A_3 : cell, invocation, creator, authority, and order are immutable. Other clauses unchanged.
Stale or malformed candidate	No durable state mutation.	(c, r)	All clauses unchanged; the attempt is τ and emits no verdict.
Crash and recovery	Discard volatile work and expose the same recovered durable image.	(c, r)	All clauses unchanged under the assumed crash-stable transaction boundary.

TABLE III
EXHAUSTIVE EVENT CLASSES AND PRESERVATION OF THE FIVE AgentSec CLAUSES.

$(\kappa, H, \Delta, \text{out}, \mathcal{A})$, $\text{Live}(H) = X_{[H.T]}$, and $I = (K, c_I, r_I)$, let $\alpha_I(I) = (K, c_I, r_I, H.\eta, H.\beta)$ and $\alpha_S(S; c, r) = (K, \text{icut}(c), r, H.\eta, \text{bind}|_{\text{Live}(H)})$. Define $I \mathcal{R} S$ iff some c, r satisfy $\text{AgentSec}(S; c, r)$ and $\alpha_I(I) = \alpha_S(S; c, r)$; write $\xRightarrow{\ell} = \tau^* \ell \tau^*$ and $\xRightarrow{\tau} = \tau^*$. Fix $I \mathcal{R} S$ with witness (c, r) . Equality of $\alpha_I(I)$ and $\alpha_S(S; c, r)$ gives both machines the same κ , current policy, normalized history, receipt ledger, outbox, result futures, semantic cut, resolved prefix, active abstract epoch, and live gate bindings.

a) History edges: The normalization equality makes both machines form the same current $\Theta_I(u)$ and request-cut token $\gamma(\Theta_I(u))$. If derivation is undefined, deterministic failed-check reflection makes both sides emit the same canonical Invalid orbit and take a state self-loop. If derivation exists but no declarative realization exists, lemma 9 and deterministic fixed-point checking make both sides emit the same canonical Reject orbit and take a state self-loop. Otherwise the ideal machine enables its history edge and, by lemma 9, the compiled checker accepts exactly the same history edge and installs the same greatest language. The compiled side may take finitely many inactive-preloading τ -steps before atomic installation. Pure compilation is outside the recovered-state LTS. The ideal side takes its single atomic edge, and both successors normalize to the same $(\kappa_u, H_u, \mathcal{A}_u)$, cut, empty prefix, active epoch, and target bindings. Deterministic allocation gives the same reply-bundle orbit, so both expose $\text{Edge}(u, [\mathcal{H}_u^+]_\cong)$. Conversely, every successful compiled installation has passed the schema, fixed-point, seal, and cut checks, so lemma 9 enables the matching ideal edge. This argument applies uniformly to all six execution edits and authenticated live extension.

b) Protected uses and concrete invocations: For a submitted invocation m , both guards compute the same $m^\circ =$

$\text{canon}_\kappa(m)$. The compiled handle check verifies the issuer, scope, occurrence, gate, cell, lineage, signed digest, current epoch, and equality $m^\circ = \text{inv}_\rho(d)$. The ideal guard requires the same normalized binding and invocation equality. Both sides then inspect the same receipted-cell set D_Δ , so they choose the same $a \in \{\text{fresh}(x, d), \text{alias}(x, d)\}$. By corollary 1 and lemma 7, ra is enabled by the compiled derivative exactly when $ra \in L^*$ in the ideal machine.

In the fresh case, both successors apply the same paired residual, append the same symbol to the global trace and branch-local cursor, advance ν , create the same cell-bound receipt, and prepare the same canonical invocation. The compiled outbox stores $(d, \text{inv}_\rho(d))$, never the caller buffer, while the ideal outbox stores (d, m°) ; the verified equality makes these records identical. In the alias case, both successors apply the same paired residual, append the same receipt-silent symbol to the global trace and branch-local cursor, and bind the occurrence to the same durable result future without changing Δ or the outbox. Both successors remove x 's live binding; because the residual frontier no longer contains x , their normalized live-binding components remain equal. In either case, lemma 14 re-establishes $\mathcal{R}$ with witness (c, ra) .

If any occurrence, gate, handle, invocation, epoch, branch, or monitor transition check fails, both normalized guards fail. Both machines emit $\text{deny}(x)$, preserve state, and remain related.

c) Use-installation races: Use and installation serialize on the same history and epoch records. If a fresh use commits first, it changes $(G, T, \Delta, \chi, \nu, \text{out})$, all of which are bound directly or through H by the cut seal, so the prepared installation becomes stale. If an alias use commits first, it changes (G, T, χ, ν) even though Δ and the outbox are unchanged, so the prepared installation again becomes stale. If

installation commits first, it atomically closes η^- , activates η^+ , refreshes every live gate and handle, and exposes replacement handles only after activation. The old fresh or alias use then fails the current-epoch guard and is denied without progress. The ideal atomic order has exactly the same two cases, so neither side admits a third race outcome.

d) Replies, releases, and silent steps: An authenticated retry, bundle retrieval, or result delivery stutters only in security-state fields and follows the same authenticated link on both sides to expose $\text{Return}(t, [v]_{\cong})$. $\text{Dispatch}(d)$ is enabled at the same outbox head and makes the same observable release update. $\text{Settle}(d, [v]_{\cong})$ is enabled for the same receipt and produces the same stable result and normalized update. Save takes matching τ -steps. Compiled preload and stale or malformed candidate attempts are implementation-only τ -steps matched by zero ideal steps because α_S omits their inactive contents and they emit no verdict. Crash is matched by a τ self-loop on each recovered-state machine by lemma 16.

Theorem 2 (Durable Agent-API realization, detailed). $\mathcal{R}$ is a divergence-insensitive weak bisimulation under obs_{api} .

Proof. The least-generated rule sets in sections IV-C and V and table III make the preceding cases exhaustive. For every step from either side, the corresponding case constructs a path with observation $\text{obs}_{\text{api}}(\ell)$ and a successor related by the witness in that matrix. The construction is symmetric because successful ideal guards and successful compiled guards were shown equivalent, while implementation-only steps preserve the common abstraction. Arbitrary repetition of silent preload, stale-candidate, Save , or crash steps is ignored, which is precisely divergence-insensitive weak matching. $\square$

H. Executable-Artifact Bounds

Above 128 outcomes, eight occurrences per outcome, 50,000 linearizations, or 100,000 completions, the executable wrapper returns $\text{ResourceLimit}(b, n)$ before search and performs no installation. This fail-closed engineering outcome is neither a semantic rejection nor an impossibility certificate. The theorem uses the uncapped finite semantic compiler.

No step in this proof infers a natural-language contract, provides fairness, implements partial fencing inside one domain, or strengthens local outbox release into remote exactly-once execution.

AI USE ACKNOWLEDGMENT

OpenAI Codex was used throughout the manuscript for initial prose drafting and revision, and in the executable artifact for code scaffolding and test generation. It also served as an interactive aid for literature discovery, counterexample search, and the discussion and refinement of the formal theory, including definitions, assumptions, theorem statements, and proof structure. The authors chose and directed the research, made all final theoretical and methodological decisions, independently verified the cited sources, theorem statements, proofs, code, and experimental results, and remain responsible for every claim, proof, result, and citation.

REFERENCES

- [1] OpenAI, “Codex App Server,” Official documentation, 2026, accessed 2026-07-31. [Online]. Available: <https://learn.chatgpt.com/docs/app-server.md>
- [2] Anthropic, “Manage sessions,” Official documentation, 2026, accessed 2026-07-31. [Online]. Available: <https://code.claude.com/docs/en/sessions>
- [3] —, “Checkpointing,” Official documentation, 2026, accessed 2026-07-31. [Online]. Available: <https://code.claude.com/docs/en/checkpointing>
- [4] C. Wang and Y. Zheng, “Fork, explore, commit: Os primitives for agentic exploration,” 2026, agentic OS Workshop 2026. [Online]. Available: <https://arxiv.org/abs/2602.08199>
- [5] T. Wu, C. Chang, L. Cao, W. Gao, and W. Wang, “Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.28138>
- [6] Y. Zhang, “Agent libos: A runtime substrate for capability-controlled self-evolving llm agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.03895>
- [7] S. Yu, D. Chong, A. Nandi, D. Soylu, J. Sun, C. D. Manning, and W. Shi, “Shepherd: Enabling programmable meta-agents via reversible agentic execution traces,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.10913>
- [8] Y. Li, S. Ping, X. Chen, X. Qi, Z. Wang, Y. Luo, and X. Zhang, “AgentGit: A version control framework for reliable and scalable LLM-powered multi-agent systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.00628>
- [9] LangChain AI, “Use time-travel,” 2026, accessed 2026-08-03. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/use-time-travel>
- [10] S. G. Patil, T. Zhang, V. Fang, N. C., R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, “GoEX: Perspectives and designs towards a runtime for autonomous LLM applications,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.06921>
- [11] E. Y. Chang and L. Geng, “SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning,” *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 4874–4886, 2025. [Online]. Available: <https://www.vldb.org/pvldb/vol18/p4874-chang.pdf>
- [12] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindshaedler, “Atomix: Timely, transactional tool use for reliable agentic workflows,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.14849>
- [13] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, “Cordon: Semantic transactions for tool-using llm agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.17573>
- [14] K. Yang, P. Li, Z. Wu, K. Xu, H. Huang, and X. Huang, “DART: Semantic recoverability for structured tool agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.23311>
- [15] M. H. de Queiroz, J. E. R. Cury, and W. M. Wonham, “Multitasking supervisory control of discrete-event systems,” *Discrete Event Dynamic Systems*, vol. 15, no. 4, pp. 375–395, 2005.
- [16] R. Malik and R. Leduc, “Generalised nonblocking,” in *Proceedings of the 9th International Workshop on Discrete Event Systems*, 2008, pp. 340–345.
- [17] B. Finkbeiner, F. Klein, and N. Metzger, “Live synthesis,” *Innovations in Systems and Software Engineering*, vol. 18, no. 3, pp. 443–454, 2022. [Online]. Available: <https://doi.org/10.1007/s11334-022-00447-5>
- [18] L. Aceto, I. Cassar, A. Francalanza, and A. Ingólfssdóttir, “On runtime enforcement via suppressions,” in *29th International Conference on Concurrency Theory (CONCUR 2018)*, ser. Leibniz International Proceedings in Informatics, vol. 118. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 34:1–34:17.
- [19] J. A. Brzozowski, “Derivatives of regular expressions,” *Journal of the ACM*, vol. 11, no. 4, pp. 481–494, 1964.
- [20] R. E. Strom and S. Yemini, “Optimistic recovery in distributed systems,” *ACM Transactions on Computer Systems*, vol. 3, no. 3, pp. 204–226, 1985.
- [21] E. Y. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A survey of rollback-recovery protocols in message-passing systems,” *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002.
- [22] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, “ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore,” 2026, coDAIM Workshop 2026. [Online]. Available: <https://arxiv.org/abs/2603.20625>
- [23] Q. Burke, A. Vahldiek-Oberwagner, M. Swift, and P. McDaniel, “It’s a feature, not a bug: Secure and auditable state rollback for confidential cloud applications,” in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, preprint first posted in 2025. [Online]. Available: <https://arxiv.org/abs/2511.13641>